4

Lab

Tìm hiểu cơ bản về Kỹ thuật Dịch ngược

Basic Reverse Engineering

Thực hành Lập trình Hệ thống

Lưu hành nội bộ

A. TỔNG QUAN

A.1 Mục tiêu

- Tìm hiểu và làm quen với Kỹ thuật Dịch ngược.
- Cài đặt và làm quen với hai công cụ miễn phí: Ghidra (dịch ngược tĩnh: disassembler + decompiler) và GDB (gỡ lỗi động: đặt breakpoint, xem thanh ghi, xem bộ nhớ khi chương trình đang chạy).

A.2 Môi trường

A.2.1 Chuẩn bị môi trường

- 01 máy Linux (máy ảo, khuyến nghị Ubuntu 64-bit) dùng để chạy file cần phân tích, cài GDB và Ghidra. Toàn bộ bài lab thực hiện trên máy này.

A.2.2 Các tập tin được cung cấp sẵn

- File **basic-reverse** là file thực thi được build cho môi trường Linux 32-bit
- Mỗi nhóm sinh viên sẽ có riêng 1 phiên bản **basic-reverse** riêng.

A.3 Liên quan

- Sinh viên cần vận dụng kiến thức về mã assembly.
- Tìm hiểu cách dùng Ghidra (Listing, Decompiler, Symbol Tree, Defined Strings, cross-reference) và các lệnh cơ bản của GDB (break, run, x, info registers, ni/si, finish).
- Tham khảo tài liệu (Mục E).

B. THỰC HÀNH

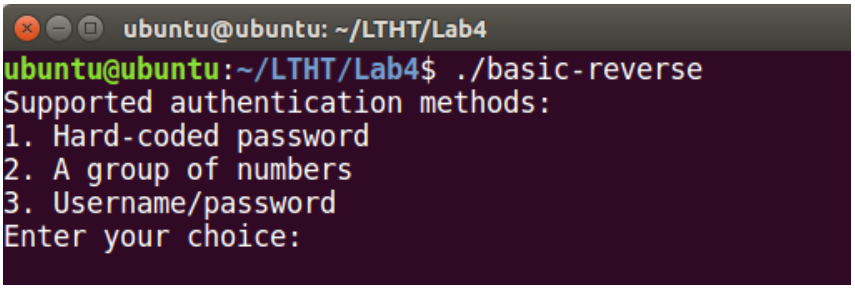
Bài thực hành này sẽ được chia làm 2 giai đoạn tương ứng với 2 yêu cầu:

- **Giai đoạn 1**: ***Thiết lập môi trường***

Trong giai đoạn này, sinh viên chuẩn bị môi trường như trong phần **A.2.** Sinh viên tự tìm hiểu tài liệu và quan sát quá trình Giảng viên hướng dẫn sơ lược.

- **Giai đoạn 2**: ***Thực hành dịch ngược file basic-reverse***

SV được cung cấp file thực thi ELF 32-bit trên Linux chạy dưới dạng command line.



File này thực thi 3 phương pháp chứng thực đơn giản yêu cầu người dùng nhập 1 chuỗi nhất định (1), 1 nhóm các số (2) hoặc một cặp username/password (3) để hoàn thành bài phân tích. Sau khi thiết lập môi trường, sinh viên áp dụng kỹ thuật dịch ngược, sử dụng Ghidra (phân tích tĩnh) và GDB (phân tích động) để phân tích file **basic-reverse** nhằm tìm ra các passphrase, cặp số hoặc cặp username/password đúng yêu cầu.

Khi nhập đúng, chương trình in ra một **flag** dạng **NT209{...}**. Mỗi option có một flag riêng. Lưu ý: **flag được mã hoá, strings không tìm ra — phải phân tích**. Flag chỉ được giải mã khi chạy chương trình với input đúng, vì khoá giải mã được tính từ chính input đó.

B.1 Yêu cầu 1 - Thiết lập môi trường

Yêu cầu 1 gồm 2 phần: (1) chuẩn bị môi trường chạy file Linux 32-bit và cài GDB; (2) cài Ghidra và tạo project phân tích file basic-reverse.

B.1.1 Thiết lập môi trường chạy file thực thi Linux 32-bit

Do file cần phân tích được build cho môi trường Linux 32-bit, để chạy được ta cần thiết lập một môi trường Linux có thể thực thi được các file thực thi 32-bit.

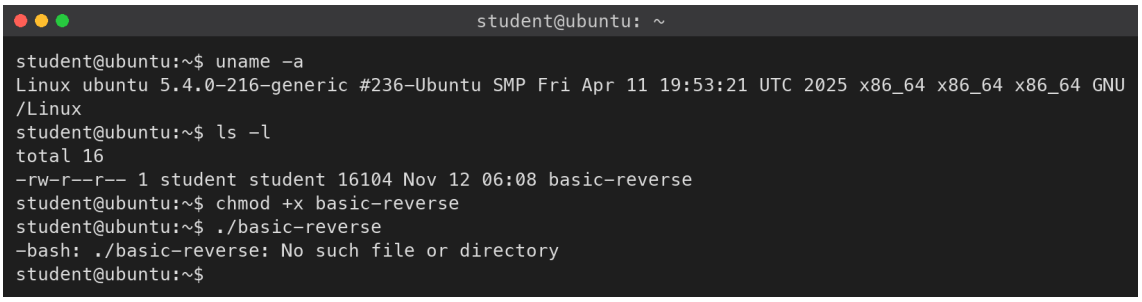
Bước 1. Kiểm tra phiên bản môi trường Linux và chạy thử file

Sao chép file basic-reverse vào máy ảo Linux, mở terminal tại thư mục chứa file và chạy các lệnh sau:

```
$ uname -a

$ chmod +x basic-reverse

$ ./basic-reverse
```



- Kết quả uname có keyword **x86_64**: máy Linux 64-bit. File basic-reverse là ELF 32-bit nên trên máy 64-bit chưa cài thư viện 32-bit sẽ báo lỗi **No such file or directory** (như hình trên) dù file vẫn tồn tại. Lỗi này là do thiếu bộ nạp /lib/ld-linux.so.2 của thư viện C 32-bit, không phải do sai đường dẫn. Cần làm tiếp Bước 2 và Bước 3.
- Nếu kết quả chỉ có i386/i686: máy 32-bit, chạy được ngay file 32-bit. Chỉ cần cài GDB ở Bước 3.

Bước 2. Thêm kiến trúc i386 (chỉ dành cho máy Linux 64-bit)

Khai báo cho trình quản lý gói dpkg biết máy chấp nhận gói của kiến trúc i386, sau đó cập nhật danh sách gói:

```
$ sudo dpkg --add-architecture i386

$ dpkg --print-foreign-architectures

$ sudo apt-get update
```

```
student@ubuntu: ~
student@ubuntu:~$ sudo dpkg --add-architecture i386
student@ubuntu:~$ dpkg --print-foreign-architectures
i386
student@ubuntu:~$ sudo apt-get update
Get:1 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Get:2 http://archive.ubuntu.com/ubuntu focal InRelease [265 kB]
Get:3 http://archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:4 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages [4368 kB]
Get:5 http://archive.ubuntu.com/ubuntu focal-backports InRelease [128 kB]
Get:6 http://archive.ubuntu.com/ubuntu focal/multiverse amd64 Packages [177 kB]
Get:7 http://security.ubuntu.com/ubuntu focal-security/restricted i386 Packages [51.0 kB]
Get:8 http://security.ubuntu.com/ubuntu focal-security/main i386 Packages [1079 kB]
Get:9 http://archive.ubuntu.com/ubuntu focal/multiverse i386 Packages [88.9 kB]
Get:10 http://security.ubuntu.com/ubuntu focal-security/universe amd64 Packages [1300 kB]
Get:11 http://archive.ubuntu.com/ubuntu focal/restricted amd64 Packages [33.4 kB]
Get:12 http://security.ubuntu.com/ubuntu focal-security/universe i386 Packages [859 kB]
Get:13 http://security.ubuntu.com/ubuntu focal-security/multiverse amd64 Packages [32.5 kB]
Get:14 http://security.ubuntu.com/ubuntu focal-security/restricted amd64 Packages [4710 kB]
Get:15 http://archive.ubuntu.com/ubuntu focal/restricted i386 Packages [10.1 kB]
Get:16 http://archive.ubuntu.com/ubuntu focal/universe i386 Packages [5999 kB]
Get:17 http://security.ubuntu.com/ubuntu focal-security/multiverse i386 Packages [8069 B]
Get:18 http://archive.ubuntu.com/ubuntu focal/main i386 Packages [930 kB]
Get:19 http://archive.ubuntu.com/ubuntu focal/universe amd64 Packages [11.3 MB]
Get:20 http://archive.ubuntu.com/ubuntu focal/main amd64 Packages [1275 kB]
Get:21 http://archive.ubuntu.com/ubuntu focal-updates/restricted amd64 Packages [4904 kB]
Get:22 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 Packages [4850 kB]
Get:23 http://archive.ubuntu.com/ubuntu focal-updates/main i386 Packages [1369 kB]
Get:24 http://archive.ubuntu.com/ubuntu focal-updates/multiverse amd64 Packages [36.2 kB]
Get:25 http://archive.ubuntu.com/ubuntu focal-updates/restricted i386 Packages [52.9 kB]
Get:26 http://archive.ubuntu.com/ubuntu focal-updates/multiverse i386 Packages [9490 B]
Get:27 http://archive.ubuntu.com/ubuntu focal-updates/universe i386 Packages [1017 kB]
Get:28 http://archive.ubuntu.com/ubuntu focal-updates/universe amd64 Packages [1588 kB]
Get:29 http://archive.ubuntu.com/ubuntu focal-backports/universe i386 Packages [15.4 kB]
Get:30 http://archive.ubuntu.com/ubuntu focal-backports/main i386 Packages [43.0 kB]
Get:31 http://archive.ubuntu.com/ubuntu focal-backports/main amd64 Packages [54.7 kB]
Get:32 http://archive.ubuntu.com/ubuntu focal-backports/universe amd64 Packages [28.4 kB]
Fetched 46.9 MB in 21s (2283 kB/s)
Reading package lists... Done
student@ubuntu:~$
```

Lệnh dpkg --print-foreign-architectures in ra **i386** nghĩa là đã thêm thành công.

Bước 3. Cài thư viện C 32-bit và GDB

```
$ sudo apt-get install -y libc6:i386 gdb file
```

```
student@ubuntu: ~
student@ubuntu:~$ sudo apt-get install -y libc6:i386 gdb file 2>&1 | tail -n 12
Setting up libbabeltrace1:amd64 (1.5.8-1build1) ...
Setting up libpython3.8-stdlib:amd64 (3.8.10-0ubuntu1~20.04.18) ...
Setting up libxml2:amd64 (2.9.10+dfsg-5ubuntu0.20.04.10) ...
Setting up libpython3.8:amd64 (3.8.10-0ubuntu1~20.04.18) ...
Setting up shared-mime-info (1.15-1) ...
Setting up gdb (9.2-0ubuntu1~20.04.2) ...
Setting up libcrypt1:i386 (1:4.4.10-10ubuntu4) ...
Setting up libgcc-s1:i386 (10.5.0-1ubuntu1~20.04) ...
Setting up libc6:i386 (2.31-0ubuntu9.18) ...
Setting up libunistring2:i386 (0.9.10-2) ...
Setting up libidn2-0:i386 (2.2.0-2) ...
Processing triggers for libc-bin (2.31-0ubuntu9.17) ...
student@ubuntu:~$
```

Hình trên chỉ hiển thị 12 dòng cuối của quá trình cài (lệnh chụp có thêm | tail -n 12 cho gọn). Cần thấy các dòng **Setting up libc6:i386** và **Setting up gdb**. Nếu gặp lỗi "Unable to locate package", chạy lại sudo apt-get update rồi cài lại.

(Tuỳ chọn) Có thể cài thêm plugin GEF hoặc pwndbg để GDB hiển thị thanh ghi, stack và mã lệnh mỗi lần dừng. Bài hướng dẫn này dùng GDB thuần nên không bắt buộc. Cách cài xem trang chủ của từng plugin (github.com/hugsy/gef, github.com/pwndbg/pwndbg).

Bước 4. Kiểm tra lại file và chạy thử

```
$ file basic-reverse

$ gdb --version | head -n 1

$ ./basic-reverse
```

```
student@ubuntu: ~
student@ubuntu:~$ file basic-reverse
basic-reverse: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpr
eter /lib/ld-linux.so.2, BuildID[sha1]=293bbb5552064108522b01c96015c77d0e48d869, for GNU/Linux 3.2.0
, not stripped
student@ubuntu:~$ gdb --version | head -n 1
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
student@ubuntu:~$ ./basic-reverse
Supported authentication methods:
1. Hard-coded password
2. A group of numbers
3. Username/password
Enter your choice: 1
Enter the hard-coded password (option 1):
hello
Sorry, wrong password. Try again!
student@ubuntu:~$
```

Lệnh file cho biết basic-reverse là **ELF 32-bit LSB executable, Intel 80386**, liên kết động với /lib/ld-linux.so.2. Nếu chương trình hiện menu như hình và trả lời được (ở đây thử option 1 với chuỗi hello nên báo sai), môi trường chạy đã sẵn sàng. Ngược lại, xem lại các bước trên.

B.1.2 Cài đặt Ghidra và tạo project phân tích

Ghidra là bộ công cụ dịch ngược mã nguồn mở do NSA phát hành, gồm disassembler (Listing) và decompiler (Decompile) chuyển mã máy thành mã giả C. Ghidra viết bằng Java nên cần JDK 21 (64-bit) trở lên.

- **Bước 1: Cài JDK 21 trên máy Linux.**

```
$ sudo apt-get install -y openjdk-21-jdk

$ java -version
```

- **Bước 2: Tải, giải nén và chạy Ghidra.**

Tải bản phát hành mới nhất (file ghidra_<phiên bản>_PUBLIC_<ngày>.zip) tại trang github.com/NationalSecurityAgency/ghidra/releases, sau đó giải nén và chạy script ghidraRun:

```
$ unzip ghidra_*_PUBLIC_*.zip

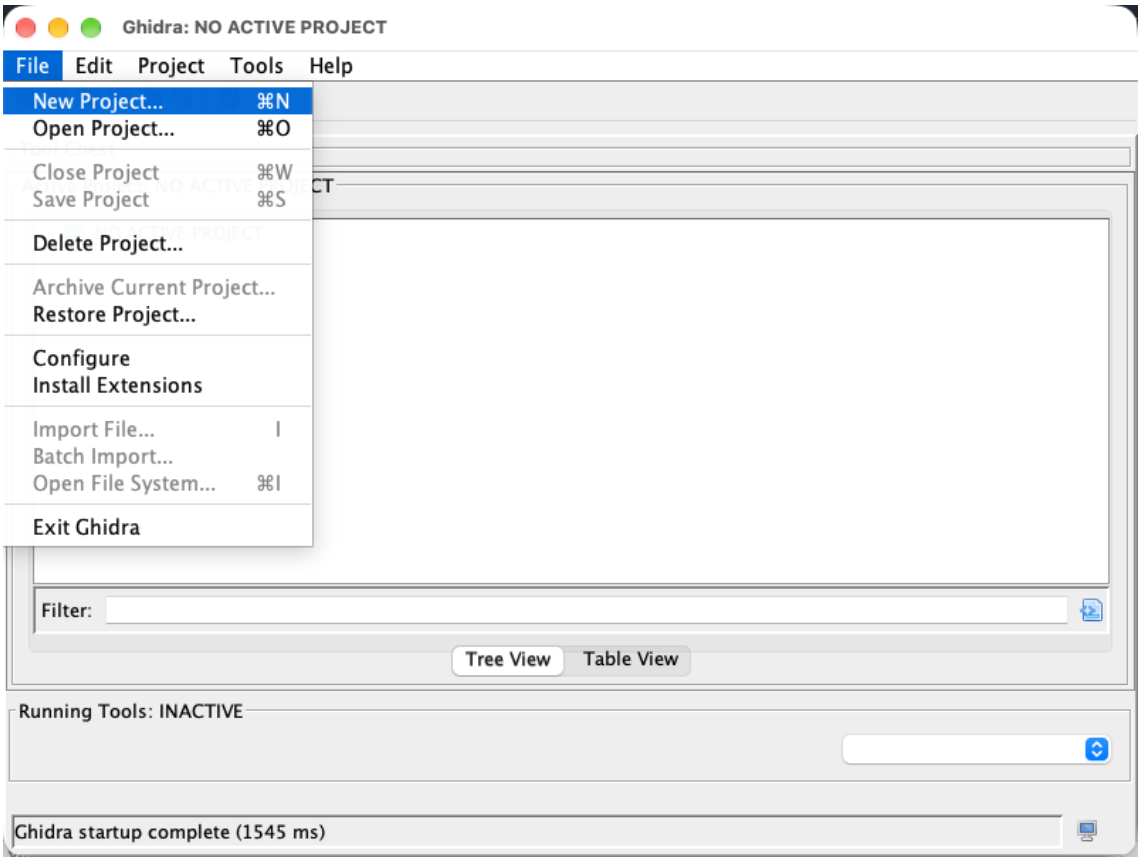
$ cd ghidra_*_PUBLIC

$ ./ghidraRun
```

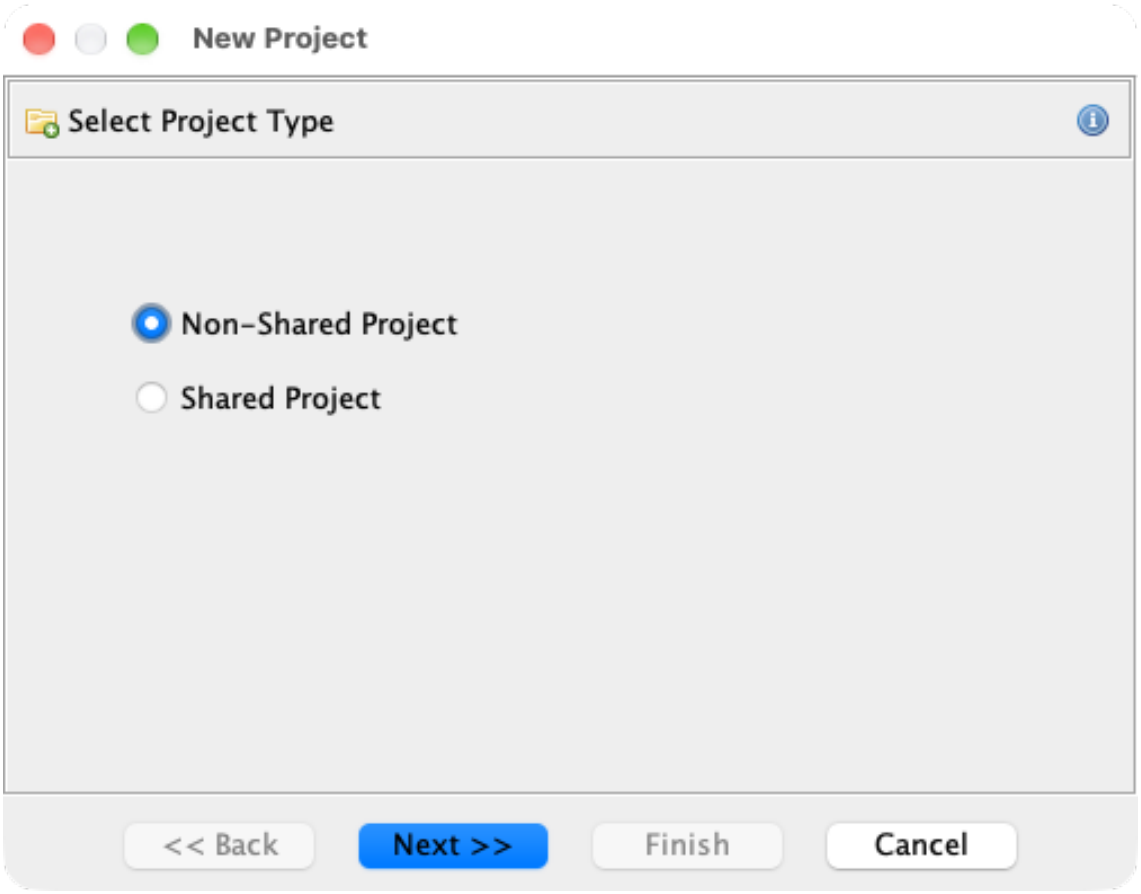
Các hình bên dưới chụp trên Ghidra 12.2. Giao diện Ghidra trên Linux giống hệt vì Ghidra là ứng dụng Java; chỉ khác đường dẫn thư mục và kiểu thanh tiêu đề cửa sổ.

- **Bước 3: Tạo project.**

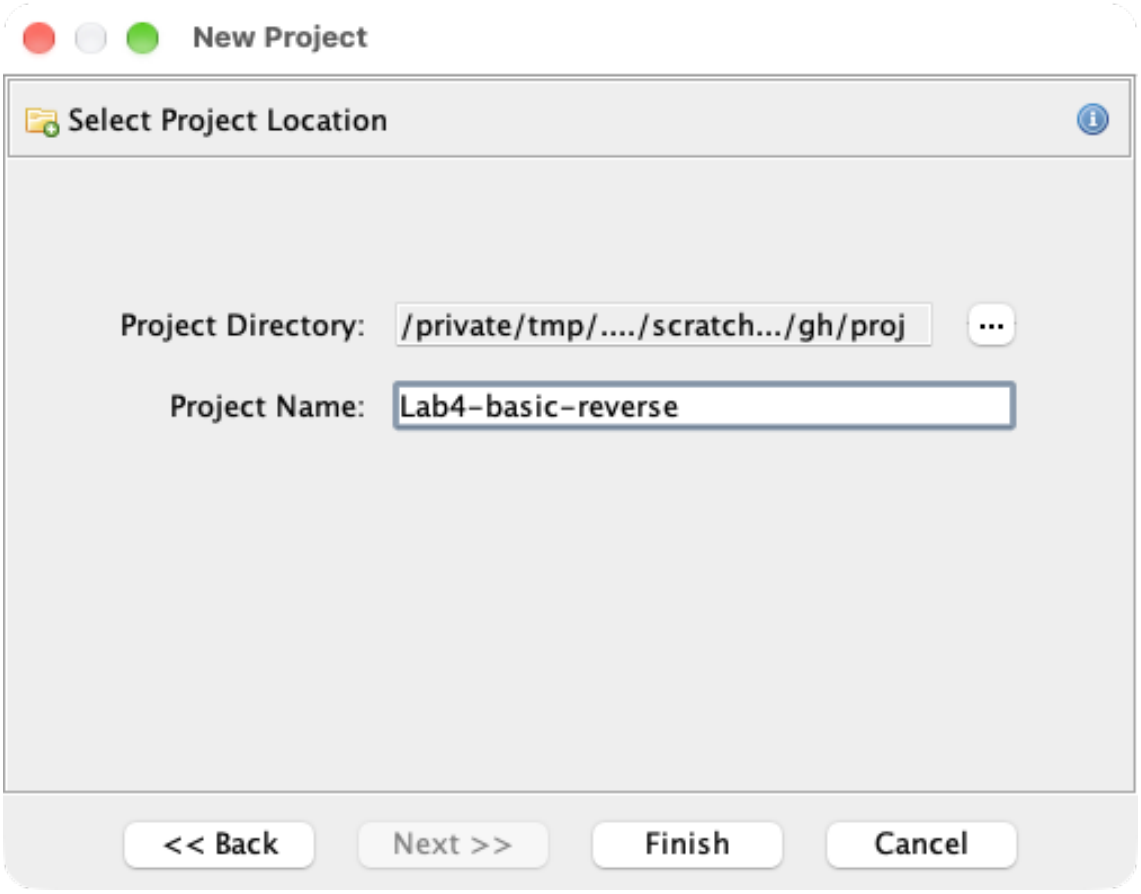
Ở cửa sổ chính của Ghidra (Project Window) chọn **File > New Project...**



Chọn **Non-Shared Project** (project lưu trên máy, không cần Ghidra Server) rồi bấm Next.

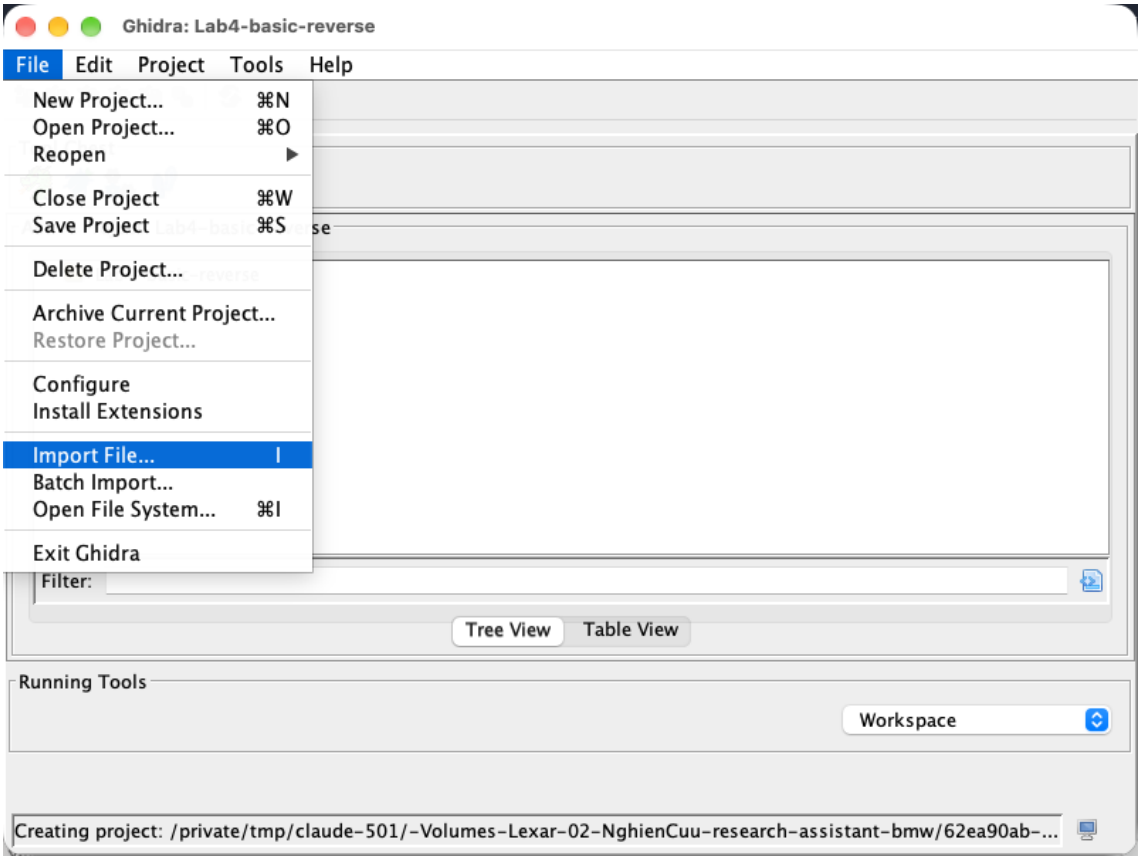


Chọn thư mục lưu project (Project Directory) và đặt tên (ví dụ Lab4-basic-reverse), bấm Finish.

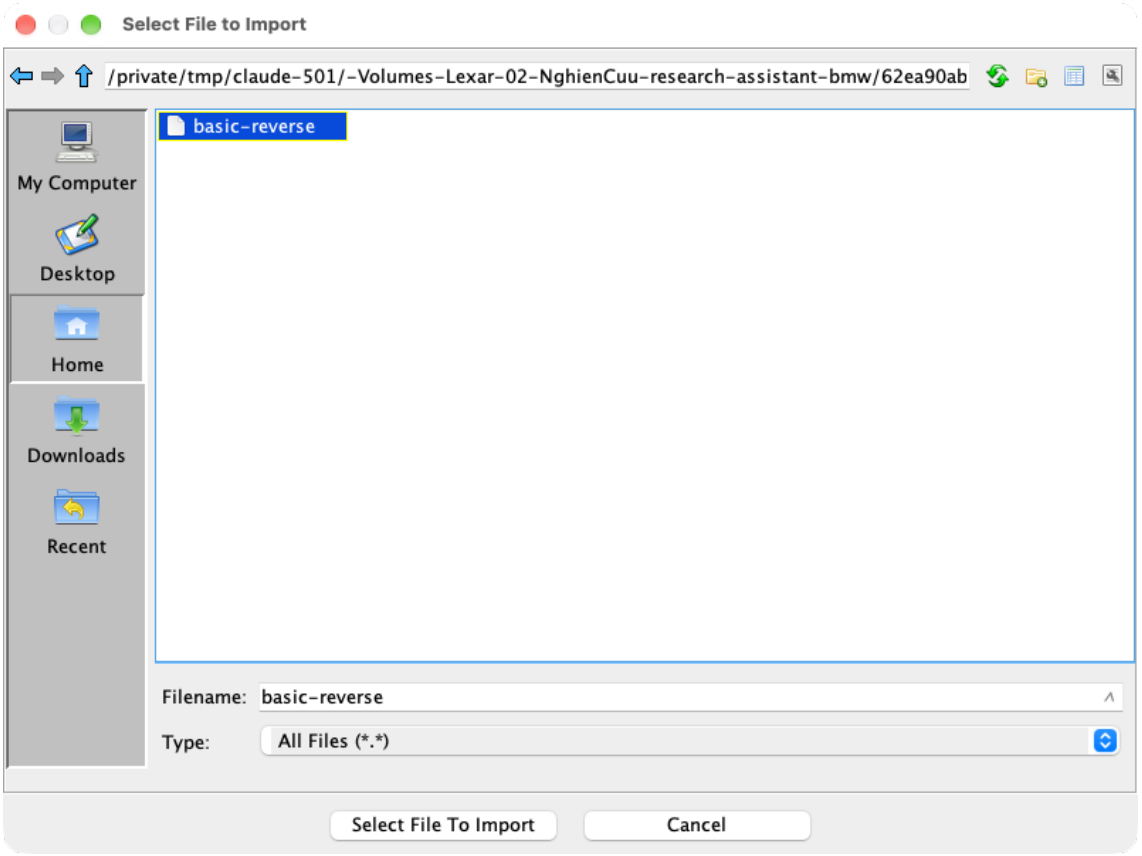


- **Bước 4: Nạp file basic-reverse vào project.**

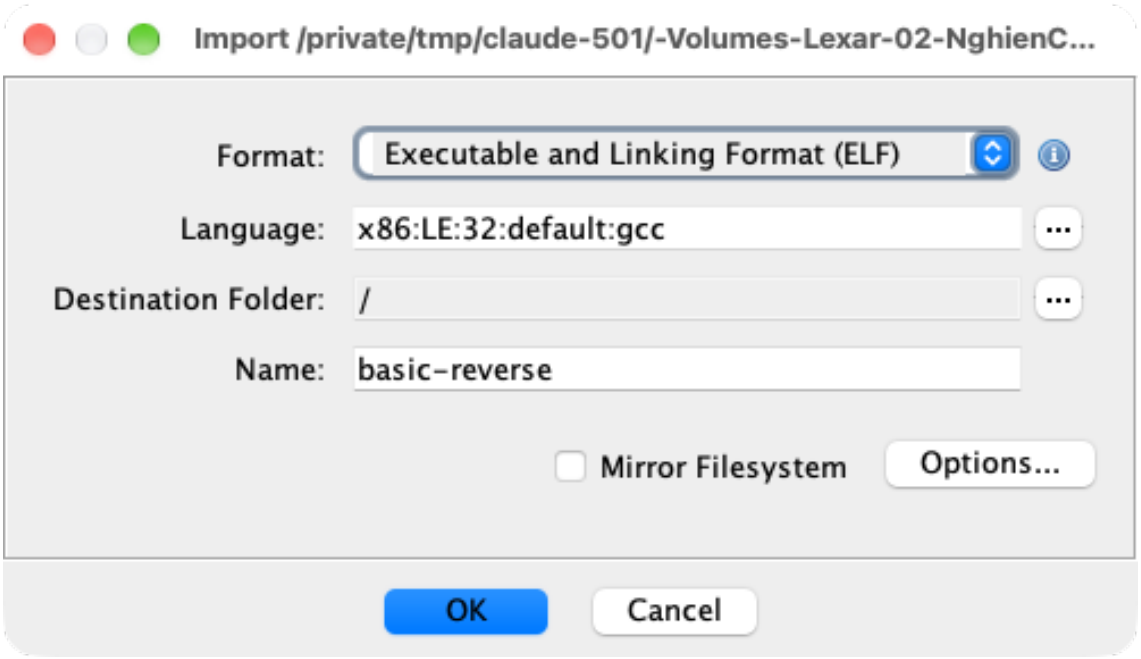
Chọn **File > Import File...** (hoặc kéo thả file vào cửa sổ project).



Chọn file basic-reverse trong hộp thoại chọn file.



Ghidra tự nhận dạng định dạng **Executable and Linking Format (ELF)** và ngôn ngữ **x86:LE:32:default** (x86 32-bit, little-endian). Giữ nguyên và bấm OK.

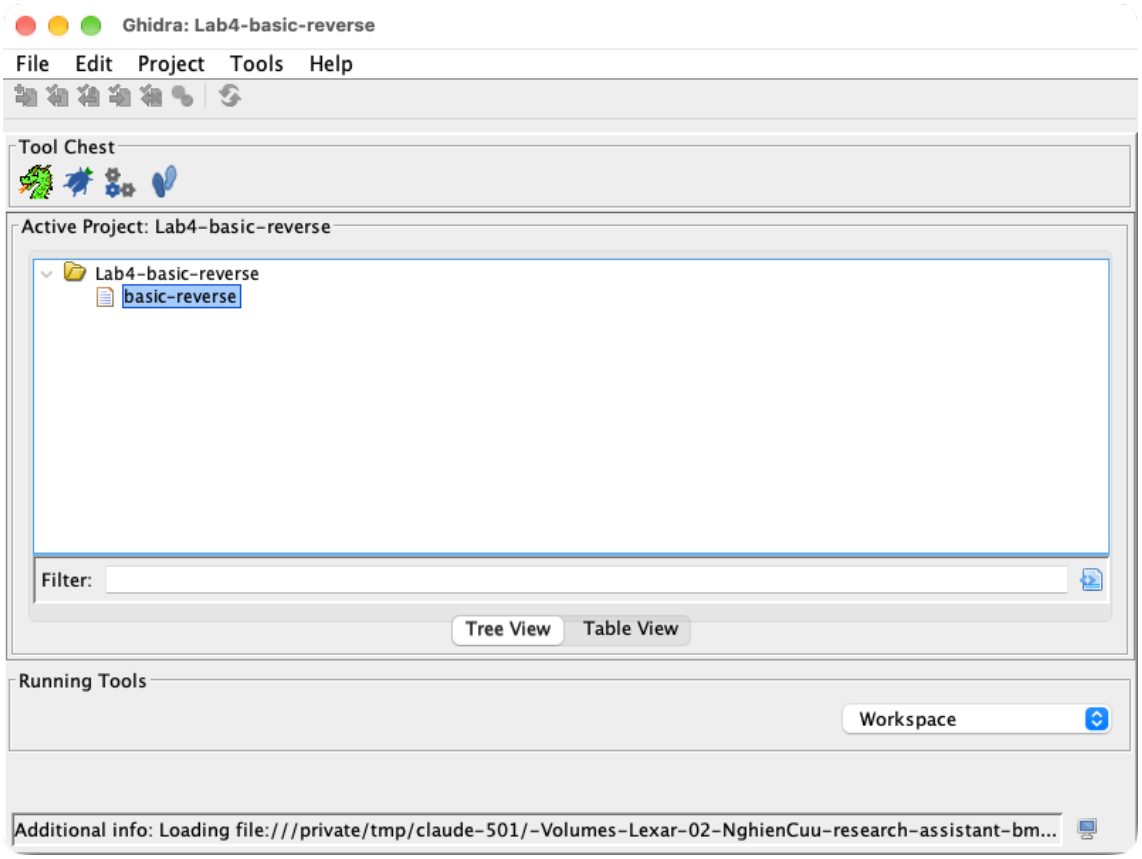


Bảng Import Results Summary tóm tắt thông tin file: định dạng ELF, trình biên dịch gcc, số hàm, số symbol... Bấm OK để đóng.

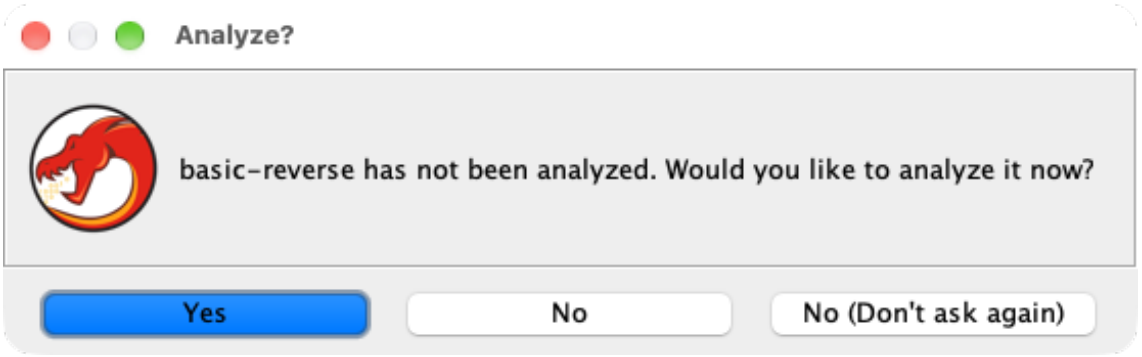


- **Bước 5: Phân tích tự động.**

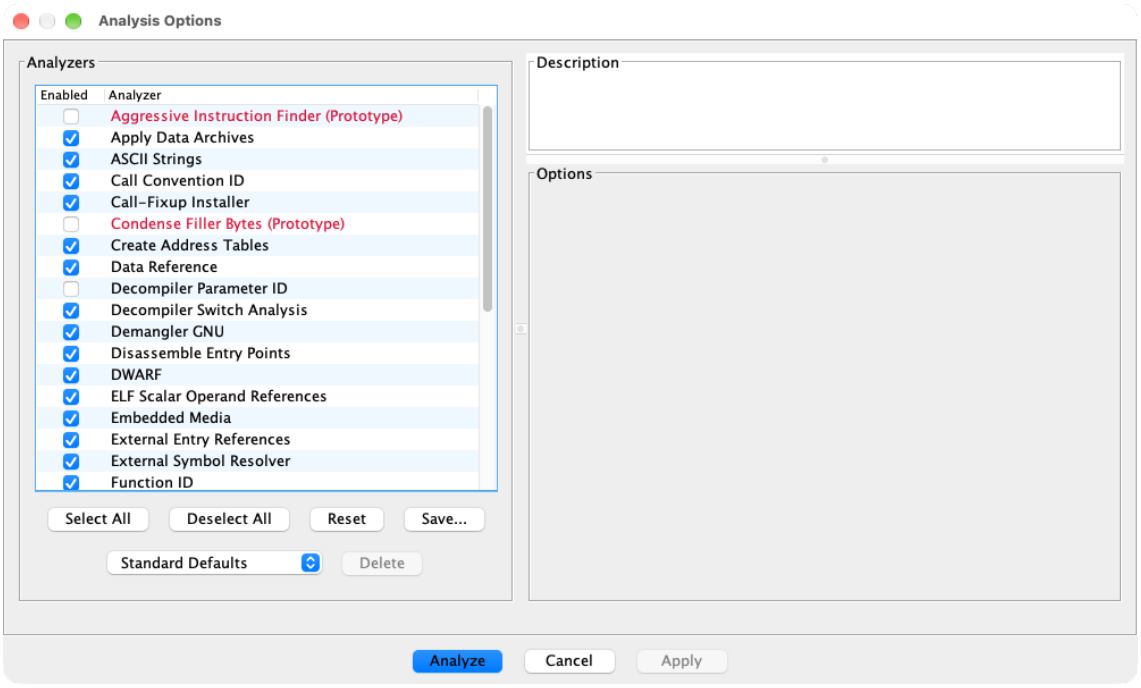
File basic-reverse đã xuất hiện trong project. Nhấp đúp vào tên file để mở bằng **CodeBrowser** (công cụ phân tích chính của Ghidra).



Lần đầu mở, Ghidra hỏi có muốn phân tích file ngay không. Chọn **Yes**.



Hộp thoại Analysis Options liệt kê các bộ phân tích (nhận diện hàm, tham chiếu, chuỗi, decompiler...). Giữ cấu hình mặc định và bấm **Analyze**. Chờ thanh tiến trình ở góc dưới phải chạy xong rồi mới bắt đầu đọc code.



B.2 Yêu cầu 2 – Thực hành phân tích file basic-reverse

B.2.1 Tổng quan về file basic-reverse

basic-reverse là một chương trình đã được lập trình sẵn, yêu cầu người dùng nhập vào một số đầu vào (chuỗi hoặc số) để đánh giá. Chương trình có 3 option khác nhau tương ứng với 3 yêu cầu đầu vào khác nhau.

- ***1. Hard-coded password*** - *Chuỗi passphrase cố định*: là một chuỗi password được định nghĩa sẵn trong chương trình. Khi chọn option này, người dùng cần nhập vào một chuỗi kết thúc bằng ký tự xuống dòng.
- ***2. A group of numbers*** – *Nhóm các số nguyên*, trong đó giữa các số sẽ có điều kiện ràng buộc nhất định được định nghĩa trong chương trình. Khi chọn option này, người dùng cần nhập nhiều số nguyên cách nhau bằng 1 dấu cách.
- ***3. Username/password*** - *Cặp username/password*: giữa username và password sẽ có những điều kiện ràng buộc nhất định được định nghĩa trong chương trình. Đầu vào yêu cầu là 1 username bất kỳ và password tương ứng của nó.

Khi nhập đúng các đầu vào như passphrase, cặp số hoặc username/password yêu cầu, chương trình sẽ in ra thông báo thành công.

Khi nhập đúng, chương trình in thêm dòng **Your flag: NT209{...}**. Flag được lưu trong file dưới dạng đã mã hoá và chỉ được giải mã bằng khoá tính từ input đúng, vì vậy lệnh strings không tìm thấy flag:

```
$ strings basic-reverse | grep "NT209{"

$ echo $?
```

```
student@ubuntu: ~/lab4
student@ubuntu:~/lab4$ strings basic-reverse | grep "NT209{"
student@ubuntu:~/lab4$ echo $?
1
student@ubuntu:~/lab4$ strings basic-reverse | grep -i hello
UIT-NT209-Hello-Ghidra
student@ubuntu:~/lab4$
```

Lệnh grep không in gì và trả mã thoát 1 (không tìm thấy). Riêng passphrase của option 1 là chuỗi so sánh trực tiếp nên vẫn thấy được bằng strings nếu đoán đúng từ khoá (dòng cuối trong hình). Đây là lý do option 1 được dùng làm ví dụ mẫu. Option 2 và 3 không có chuỗi đáp án lưu sẵn: phải đọc thuật toán.

B.2.2 Yêu cầu thực hành

Yêu cầu 1. Phân tích và tìm ***passphrase cố định*** của **basic-reverse** với option 1. Báo cáo phương pháp phân tích, input tìm được, flag nhận được và hình ảnh minh chứng chạy file.

Yêu cầu 2. Phân tích và tìm ***nhóm các số nguyên*** của **basic-reverse** với option 2. Báo cáo phương pháp phân tích, input tìm được, flag nhận được và hình ảnh minh chứng chạy file.

Yêu cầu 3. Phân tích, tìm **cặp *username/password*** phù hợp của **basic-reverse** với option 3. Báo cáo phương pháp phân tích, input tìm được, flag nhận được và hình ảnh minh chứng chạy file.

Lưu ý bắt buộc: ***username*** được tạo từ MSSV của các thành viên trong nhóm.

- Nhóm **3 sinh viên:** sắp xếp 3 MSSV theo thứ tự tăng dần rồi lấy 3 số cuối nối nhau. Ví dụ 23520013 < 24520123 < 24521021 sẽ có username là **013123021**.
- Nhóm **2 sinh viên:** sắp xếp 2 MSSV theo thứ tự tăng dần rồi lấy 4 số cuối nối nhau bằng dấu "-". Ví dụ 24520013 < 24520123 sẽ có username là **0013-0123**.
- Nhóm có **1 sinh viên** có MSSV là 2452xxxx thì username là **2452-xxxx**.

B.2.3 Hướng dẫn cách thực hiện phân tích file basic-reverse

- **Bước 1: Thực thi thử file basic-reverse trên Linux**

```
./basic-reverse
```

Bước đầu ta cần lựa chọn 1 trong 3 phương pháp chứng thực bằng cách nhập số 1, 2 hoặc 3. Tương ứng với từng lựa chọn chương trình sẽ in ra yêu cầu nhập input tương ứng, giả sử bên dưới lựa chọn option 1 là dùng passphrase cố định thì sẽ được yêu cầu nhập 1 chuỗi. Vì ban đầu chưa biết input cần nhập là gì nên có thể nhập một chuỗi bất kì. Lúc này chương trình thông báo sai và thoát.

```
student@ubuntu: ~/lab4
student@ubuntu:~/lab4$ ./basic-reverse
Supported authentication methods:
1. Hard-coded password
2. A group of numbers
3. Username/password
Enter your choice: 1
Enter the hard-coded password (option 1):
UIT-NT209-Hello-Ghidra
Congratulation! You got the hard-coded password!
Your flag: NT209{h4rdc0d3d_str1ngs_4r3_n0t_s3cr3ts}
student@ubuntu:~/lab4$
```

Hình trên là kết quả khi đã nhập đúng passphrase của option 1 (xem phần phân tích bên dưới): chương trình in thông báo thành công và flag. Khi nhập sai, chương trình in "Sorry, wrong password. Try again!" và thoát.

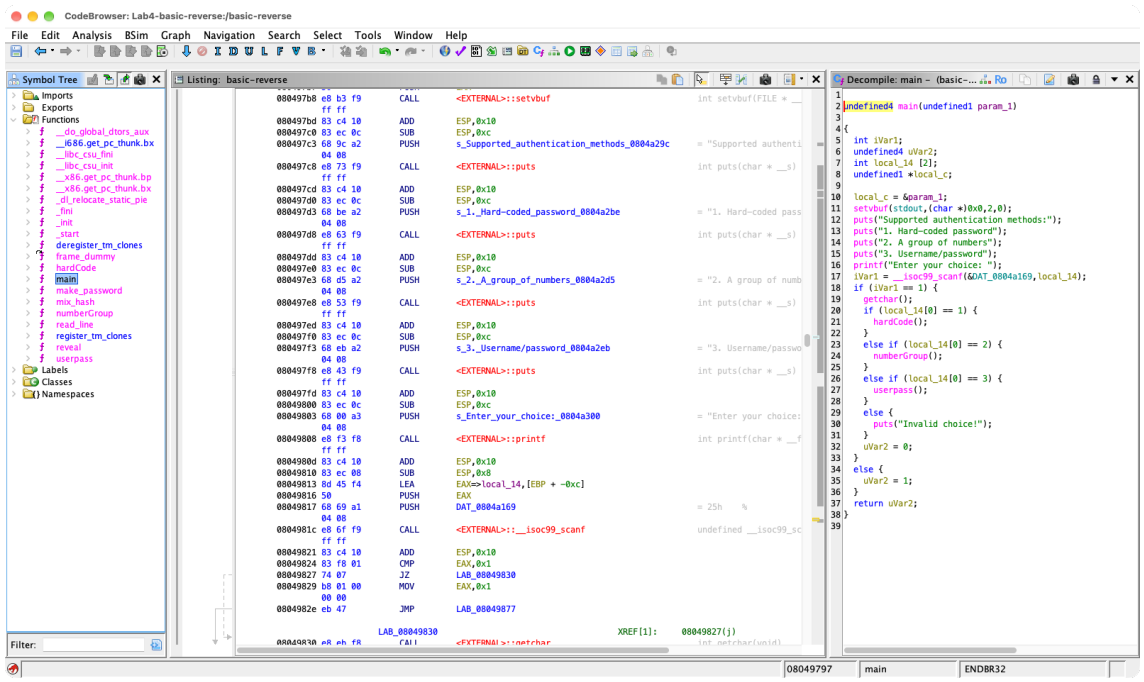
- **Bước 2: Phân tích file basic-reverse với Ghidra**

- **Xác định hàm cần quan tâm**

Sau khi phân tích tự động xong, cửa sổ CodeBrowser gồm các khung chính:

+ **Symbol Tree** (trái): danh sách hàm (Functions), nhãn, biến toàn cục. Mở Functions để thấy main, hardCode, numberGroup, userpass và các hàm thư viện như puts, printf, strcmp.
+ **Listing** (giữa): mã assembly kèm địa chỉ, byte lệnh, nhãn và chú thích tham chiếu (XREF).
+ **Decompile** (phải): mã giả C của hàm đang chọn. Nhấp vào một dòng ở Decompile thì Listing tô sáng các lệnh assembly tương ứng và ngược lại.

Trong Symbol Tree, mở Functions và nhấp vào **main**. File basic-reverse là ELF 32-bit không PIE nên địa chỉ trong Ghidra trùng với địa chỉ khi chạy trong GDB (ví dụ main bắt đầu tại 0x08049797).

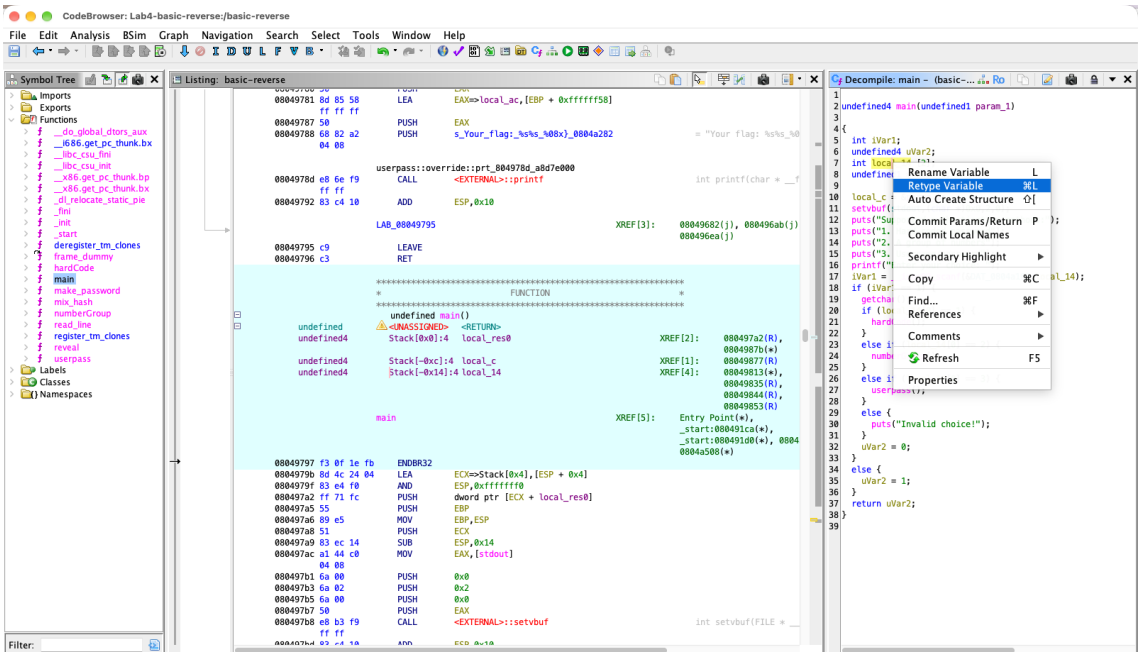


- **Phân tích code của hàm main**

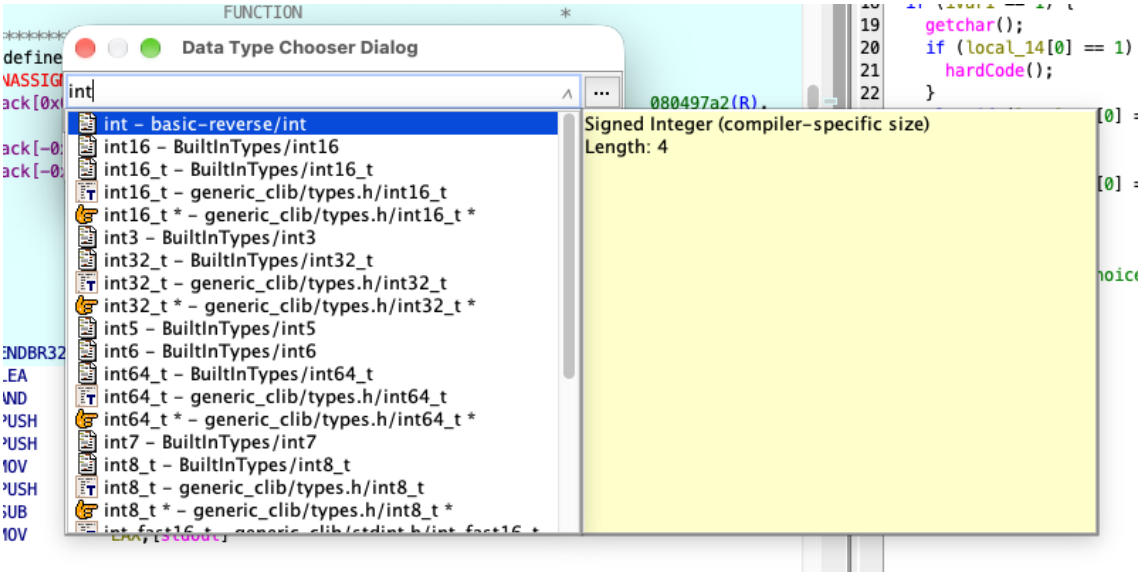
Đọc song song Listing và Decompile của main:

+ Các lệnh đầu hàm (push ebp; mov ebp,esp; sub esp,...) tạo stack frame, có thể bỏ qua.
+ Các cặp lệnh **push <địa chỉ chuỗi>** rồi **call puts** / **call printf** in menu. Decompile hiển thị thành puts("Supported authentication methods:"), ... printf("Enter your choice: ").
+ Lệnh **lea eax,[ebp-0xc]** lấy địa chỉ biến cục bộ, rồi **push 0x804a169** (chuỗi định dạng "%d") và **call __isoc99_scanf**: đọc một số nguyên vào biến tại [ebp-0xc]. Hai tham số được push theo thứ tự ngược với khai báo C: scanf("%d", &bien).
+ Sau đó giá trị tại [ebp-0xc] được nạp vào eax và so sánh lần lượt với 1, 2, 3 (**cmp eax,0x1** ... **cmp eax,0x3**); mỗi nhánh gọi một hàm: hardCode, numberGroup, userpass. Đây chính là đoạn kiểm tra lựa chọn option.

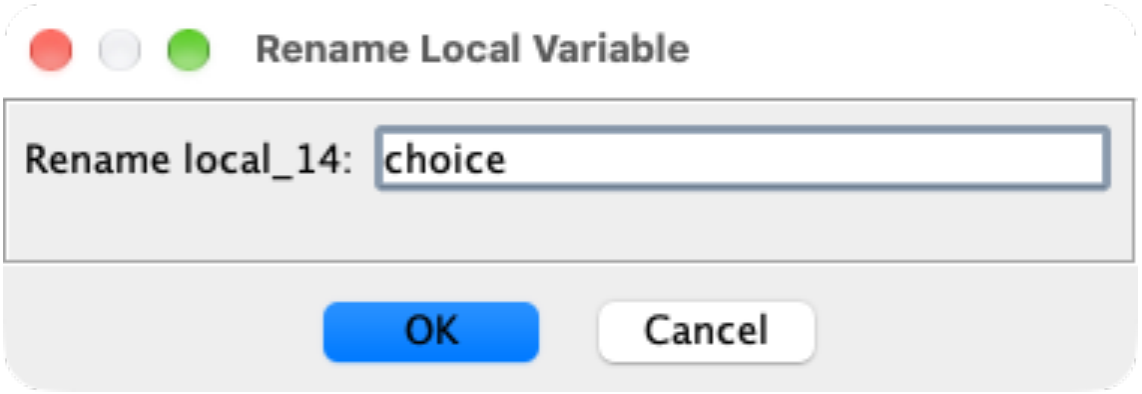
Decompiler đặt tên biến tự động (local_14) và đoán kiểu int[2]. Ta sửa lại cho dễ đọc. Nhấp chuột phải vào local_14 trong cửa sổ Decompile, chọn **Retype Variable** (phím tắt Ctrl+L):



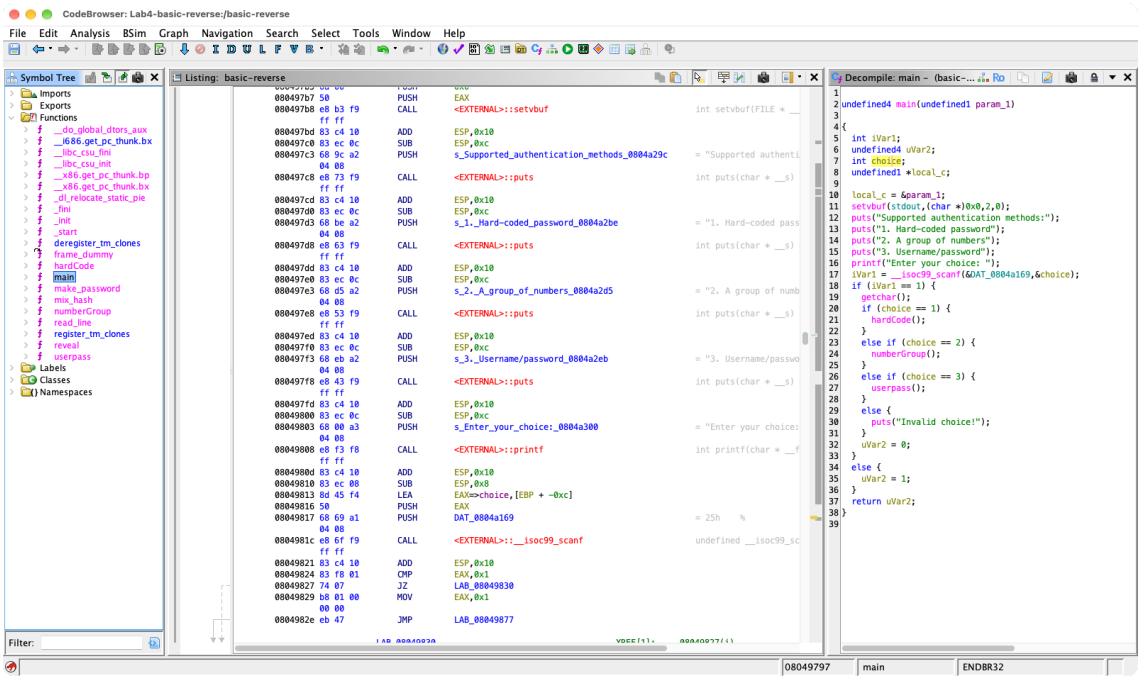
Gõ **int** và chọn kiểu int trong danh sách gợi ý, bấm OK.



Tiếp tục nhấp chuột phải vào biến, chọn **Rename Variable** (phím tắt L) và đặt tên **choice**:

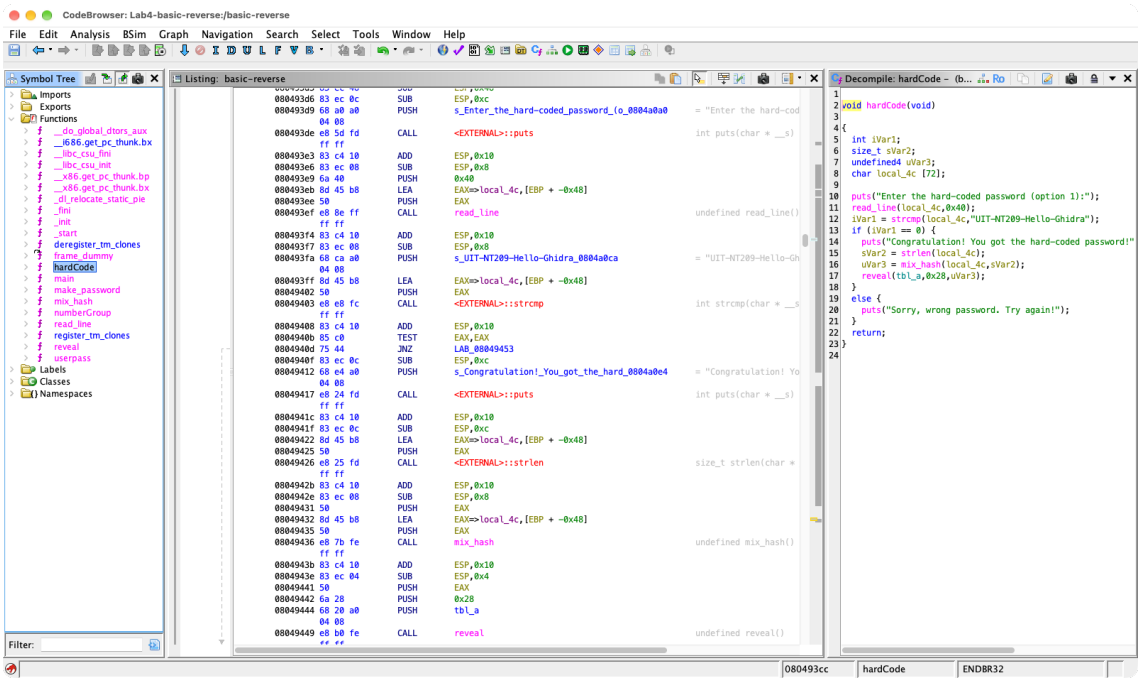


Mã giả trở nên rõ ràng: scanf("%d", &choice) rồi if (choice == 1) hardCode(); else if (choice == 2) numberGroup(); ... Đổi tên và đổi kiểu chỉ thay đổi cách Ghidra hiển thị trong project, không sửa file basic-reverse.



- **Phân tích hàm hardCode (option 1)**

Nhấp đúp vào tên hàm hardCode (trong Decompile hoặc Symbol Tree) để nhảy tới hàm:



+ puts(...) in yêu cầu nhập passphrase, read_line(local_4c, 0x40) đọc tối đa 0x40 = 64 ký tự vào mảng cục bộ local_4c (tương ứng [ebp-0x48] trong Listing).
+ strcmp(local_4c, "UIT-NT209-Hello-Ghidra") so sánh input với một chuỗi hằng. Trong Listing là hai lệnh **push 0x804a0ca** (địa chỉ chuỗi hằng) và **push eax** (eax = địa chỉ [ebp-0x48]) trước **call strcmp** tại 0x08049403.

+ strcmp trả về 0 khi hai chuỗi bằng nhau (lệnh test eax,eax; jne ... nhảy sang nhánh báo sai khi eax khác 0). Nhánh đúng in thông báo thành công, gọi mix_hash(input, strlen(input)) để tính khoá, rồi reveal(...) giải mã và in flag.

Như vậy passphrase của option 1 là chuỗi hằng được so sánh trong strcmp. Phần hướng dẫn sử dụng Ghidra ở mục C trình bày thêm cách đổi tên biến local_4c, tìm chuỗi và tham chiếu chéo.

- **Bước 3: Kiểm chứng bằng GDB (phân tích động)**

Ghidra cho biết "chương trình định làm gì"; GDB cho thấy "chương trình thực sự làm gì" khi chạy: giá trị tham số, thanh ghi, kết quả trả về. Mở file bằng GDB và chuyển cú pháp assembly sang Intel cho giống Ghidra:

```
$ gdb -q ./basic-reverse

(gdb) set disassembly-flavor intel

(gdb) info functions
```

```
student@ubuntu: ~/lab4
student@ubuntu:~/lab4$ gdb -q ./basic-reverse
Reading symbols from ./basic-reverse...
(No debugging symbols found in ./basic-reverse)
(gdb) set disassembly-flavor intel
(gdb) info functions
All defined functions:

Non-debugging symbols:
0x08049000  _init
0x080490f0  strcmp@plt
0x08049100  printf@plt
0x08049110  strcspn@plt
0x08049120  getchar@plt
0x08049130  fgets@plt
0x08049140  puts@plt
0x08049150  strlen@plt
0x08049160  __libc_start_main@plt
0x08049170  setvbuf@plt
0x08049180  snprintf@plt
0x08049190  __isoc99_scanf@plt
0x080491a0  _start
0x080491e0  _dl_relocate_static_pie
0x080491f0  __x86.get_pc_thunk.bx
0x08049200  deregister_tm_clones
0x08049240  register_tm_clones
0x08049280  __do_global_dtors_aux
0x080492b0  frame_dummy
0x080492b6  mix_hash
0x080492fe  reveal
0x08049382  read_line
0x080493cc  hardCode
0x08049466  numberGroup
0x08049565  make_password
0x08049605  userpass
0x08049797  main
0x08049880  __libc_csu_init
0x080498f0  __libc_csu_fini
0x080498f5  __x86.get_pc_thunk.bp
0x080498fc  _fini
(gdb)
```

info functions liệt kê các hàm có symbol cùng địa chỉ: main, hardCode, numberGroup, userpass, mix_hash, reveal... và các hàm thư viện qua PLT (puts@plt, strcmp@plt...). Các địa chỉ này trùng với Symbol Tree của Ghidra.

```
(gdb) disassemble main
```

```
student@ubuntu: ~/lab4
(gdb) disassemble main
Dump of assembler code for function main:
   0x08049797 <+0>:     endbr32
   0x0804979b <+4>:     lea    ecx,[esp+0x4]
   0x0804979f <+8>:     and    esp,0xfffffff0
   0x080497a2 <+11>:    push   DWORD PTR [ecx-0x4]
   0x080497a5 <+14>:    push   ebp
   0x080497a6 <+15>:    mov    ebp,esp
   0x080497a8 <+17>:    push   ecx
   0x080497a9 <+18>:    sub    esp,0x14
   0x080497ac <+21>:    mov    eax,ds:0x804c044
   0x080497b1 <+26>:    push   0x0
   0x080497b3 <+28>:    push   0x2
   0x080497b5 <+30>:    push   0x0
   0x080497b7 <+32>:    push   eax
   0x080497b8 <+33>:    call   0x8049170 <setvbuf@plt>
   0x080497bd <+38>:    add    esp,0x10
   0x080497c0 <+41>:    sub    esp,0xc
   0x080497c3 <+44>:    push   0x804a29c
   0x080497c8 <+49>:    call   0x8049140 <puts@plt>
   0x080497cd <+54>:    add    esp,0x10
   0x080497d0 <+57>:    sub    esp,0xc
   0x080497d3 <+60>:    push   0x804a2be
   0x080497d8 <+65>:    call   0x8049140 <puts@plt>
   0x080497dd <+70>:    add    esp,0x10
   0x080497e0 <+73>:    sub    esp,0xc
   0x080497e3 <+76>:    push   0x804a2d5
   0x080497e8 <+81>:    call   0x8049140 <puts@plt>
   0x080497ed <+86>:    add    esp,0x10
   0x080497f0 <+89>:    sub    esp,0xc
   0x080497f3 <+92>:    push   0x804a2eb
   0x080497f8 <+97>:    call   0x8049140 <puts@plt>
   0x080497fd <+102>:   add    esp,0x10
   0x08049800 <+105>:   sub    esp,0xc
   0x08049803 <+108>:   push   0x804a300
   0x08049808 <+113>:   call   0x8049100 <printf@plt>
   0x0804980d <+118>:   add    esp,0x10
   0x08049810 <+121>:   sub    esp,0x8
   0x08049813 <+124>:   lea    eax,[ebp-0xc]
   0x08049816 <+127>:   push   eax
   0x08049817 <+128>:   push   0x804a169
   0x0804981c <+133>:   call   0x8049190 <__isoc99_scanf@plt>
   0x08049821 <+138>:   add    esp,0x10
   0x08049824 <+141>:   cmp    eax,0x1
   0x08049827 <+144>:   je     0x8049830 <main+153>
   0x08049829 <+146>:   mov    eax,0x1
   0x0804982e <+151>:   jmp    0x8049877 <main+224>
   0x08049830 <+153>:   call   0x8049120 <getchar@plt>
   0x08049835 <+158>:   mov    eax,DWORD PTR [ebp-0xc]
   0x08049838 <+161>:   cmp    eax,0x1
   0x0804983b <+164>:   jne    0x8049844 <main+173>
   0x0804983d <+166>:   call   0x80493cc <hardCode>
   0x08049842 <+171>:   jmp    0x8049872 <main+219>
   0x08049844 <+173>:   mov    eax,DWORD PTR [ebp-0xc]
   0x08049847 <+176>:   cmp    eax,0x2
   0x0804984a <+179>:   jne    0x8049853 <main+188>
   0x0804984c <+181>:   call   0x8049466 <numberGroup>
   0x08049851 <+186>:   jmp    0x8049872 <main+219>
   0x08049853 <+188>:   mov    eax,DWORD PTR [ebp-0xc]
   0x08049856 <+191>:   cmp    eax,0x3
   0x08049859 <+194>:   jne    0x8049862 <main+203>
   0x0804985b <+196>:   call   0x8049605 <userpass>
   0x08049860 <+201>:   jmp    0x8049872 <main+219>
   0x08049862 <+203>:   sub    esp,0xc
   0x08049865 <+206>:   push   0x804a314
   0x0804986a <+211>:   call   0x8049140 <puts@plt>
   0x0804986f <+216>:   add    esp,0x10
   0x08049872 <+219>:   mov    eax,0x0
   0x08049877 <+224>:   mov    ecx,DWORD PTR [ebp-0x4]
   0x0804987a <+227>:   leave
   0x0804987b <+228>:   lea    esp,[ecx-0x4]
   0x0804987e <+231>:   ret
End of assembler dump.
(gdb)
```

disassemble <tên hàm> in mã assembly của hàm, mỗi dòng có địa chỉ và độ lệch <+n> tính từ đầu hàm. Kết quả giống Listing của Ghidra. Dùng **x/s <địa chỉ>** để xem chuỗi tại các địa chỉ được push:

```
(gdb) x/s 0x804a29c

(gdb) x/s 0x804a300

(gdb) x/s 0x804a169
```

```
student@ubuntu: ~/lab4
(gdb) x/s 0x804a29c
0x804a29c:      "Supported authentication methods:"
(gdb) x/s 0x804a300
0x804a300:      "Enter your choice: "
(gdb) x/s 0x804a169
0x804a169:      "%d"
(gdb)
```

Lệnh x (examine) có dạng **x/<số lượng><định dạng><kích thước> <địa chỉ>**. Định dạng hay dùng: s (chuỗi), x (hex), d (số nguyên có dấu), i (lệnh). Kích thước: b (1 byte), h (2), w (4), g (8). Ví dụ x/2wx $esp in 2 word 4 byte dạng hex tại đỉnh stack.

```
(gdb) disassemble hardCode
```

```
student@ubuntu: ~/lab4
(gdb) disassemble hardCode
Dump of assembler code for function hardCode:
   0x080493cc <+0>:     endbr32
   0x080493d0 <+4>:     push   ebp
   0x080493d1 <+5>:     mov    ebp,esp
   0x080493d3 <+7>:     sub    esp,0x48
   0x080493d6 <+10>:    sub    esp,0xc
   0x080493d9 <+13>:    push   0x804a0a0
   0x080493de <+18>:    call   0x8049140 <puts@plt>
   0x080493e3 <+23>:    add    esp,0x10
   0x080493e6 <+26>:    sub    esp,0x8
   0x080493e9 <+29>:    push   0x40
   0x080493eb <+31>:    lea    eax,[ebp-0x48]
   0x080493ee <+34>:    push   eax
   0x080493ef <+35>:    call   0x8049382 <read_line>
   0x080493f4 <+40>:    add    esp,0x10
   0x080493f7 <+43>:    sub    esp,0x8
   0x080493fa <+46>:    push   0x804a0ca
   0x080493ff <+51>:    lea    eax,[ebp-0x48]
   0x08049402 <+54>:    push   eax
   0x08049403 <+55>:    call   0x80490f0 <strcmp@plt>
   0x08049408 <+60>:    add    esp,0x10
   0x0804940b <+63>:    test   eax,eax
   0x0804940d <+65>:    jne    0x8049453 <hardCode+135>
   0x0804940f <+67>:    sub    esp,0xc
   0x08049412 <+70>:    push   0x804a0e4
   0x08049417 <+75>:    call   0x8049140 <puts@plt>
   0x0804941c <+80>:    add    esp,0x10
   0x0804941f <+83>:    sub    esp,0xc
   0x08049422 <+86>:    lea    eax,[ebp-0x48]
   0x08049425 <+89>:    push   eax
   0x08049426 <+90>:    call   0x8049150 <strlen@plt>
   0x0804942b <+95>:    add    esp,0x10
   0x0804942e <+98>:    sub    esp,0x8
   0x08049431 <+101>:   push   eax
   0x08049432 <+102>:   lea    eax,[ebp-0x48]
   0x08049435 <+105>:   push   eax
   0x08049436 <+106>:   call   0x80492b6 <mix_hash>
   0x0804943b <+111>:   add    esp,0x10
   0x0804943e <+114>:   sub    esp,0x4
   0x08049441 <+117>:   push   eax
   0x08049442 <+118>:   push   0x28
   0x08049444 <+120>:   push   0x804a020
   0x08049449 <+125>:   call   0x80492fe <reveal>
   0x0804944e <+130>:   add    esp,0x10
   0x08049451 <+133>:   jmp    0x8049463 <hardCode+151>
   0x08049453 <+135>:   sub    esp,0xc
   0x08049456 <+138>:   push   0x804a118
   0x0804945b <+143>:   call   0x8049140 <puts@plt>
   0x08049460 <+148>:   add    esp,0x10
   0x08049463 <+151>:   nop
   0x08049464 <+152>:   leave
   0x08049465 <+153>:   ret
End of assembler dump.
(gdb)
```

Kết hợp với Ghidra: lệnh **call strcmp** nằm tại địa chỉ **0x08049403** (cột địa chỉ trong Listing của Ghidra hoặc dòng hardCode+55 trong GDB). Đặt breakpoint ngay tại lệnh call này để xem hai tham số trước khi strcmp chạy. Cú pháp **break *<địa chỉ>** (có dấu *) đặt breakpoint tại một địa chỉ lệnh cụ thể; break <tên hàm> thì dừng ở đầu hàm.

```
(gdb) break *0x08049403

(gdb) run
```

```
student@ubuntu: ~/lab4
(gdb) break *0x08049403
Breakpoint 1 at 0x8049403
(gdb) run
Starting program: /home/student/lab4/basic-reverse
Supported authentication methods:
1. Hard-coded password
2. A group of numbers
3. Username/password
Enter your choice: 1
Enter the hard-coded password (option 1):
hello

Breakpoint 1, 0x08049403 in hardCode ()
(gdb)
```

Chạy chương trình, chọn option 1 và nhập thử chuỗi bất kỳ (hello). Chương trình dừng tại breakpoint, trước khi gọi strcmp. Lúc này hai tham số của strcmp đã được push lên stack: [esp] là tham số thứ nhất, [esp+4] là tham số thứ hai (quy ước cdecl của IA32).

```
(gdb) info registers eax esp ebp eip

(gdb) x/2wx $esp

(gdb) x/s *(char )$esp

(gdb) x/s *(char )($esp+4)
```

```
student@ubuntu: ~/lab4
(gdb) info registers eax esp ebp eip
eax            0xffffd690          -10608
esp            0xffffd680          0xffffd680
ebp            0xffffd6d8          0xffffd6d8
eip            0x8049403           0x8049403 <hardCode+55>
(gdb) x/2wx $esp
0xffffd680:     0xffffd690      0x0804a0ca
(gdb) x/s *(char **)$esp
0xffffd690:     "hello"
(gdb) x/s *(char **)($esp+4)
0x804a0ca:      "UIT-NT209-Hello-Ghidra"
(gdb)
```

+ **info registers** (viết tắt i r) xem giá trị thanh ghi; eip = 0x8049403 <hardCode+55> đúng vị trí breakpoint.
+ **x/2wx $esp** cho 2 con trỏ: 0xffffd690 (mảng chứa input trên stack) và 0x0804a0ca (chuỗi hằng trong vùng dữ liệu).

+ **x/s *(char**)$esp **đọc con trỏ tại [esp] rồi in chuỗi nó trỏ tới: "hello" là input vừa nhập. Tham số thứ hai là chuỗi passphrase:** "UIT-NT209-Hello-Ghidra"**.**

Dùng **ni** (next instruction: chạy 1 lệnh, không đi vào hàm con) để thực thi lệnh call strcmp rồi xem giá trị trả về trong eax:

```
(gdb) ni
(gdb) x/3i $eip
(gdb) p $eax
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) x/i $eip
(gdb) continue
```

```
student@ubuntu: ~/lab4
(gdb) ni
0x08049408 in hardCode ()
(gdb) x/3i $eip
=> 0x8049408 <hardCode+60>:     add    esp,0x10
   0x804940b <hardCode+63>:     test   eax,eax
   0x804940d <hardCode+65>:     jne    0x8049453 <hardCode+135>
(gdb) p $eax
$1 = 1
(gdb) ni
0x0804940b in hardCode ()
(gdb) ni
0x0804940d in hardCode ()
(gdb) ni
0x08049453 in hardCode ()
(gdb) x/i $eip
=> 0x8049453 <hardCode+135>:    sub    esp,0xc
(gdb) continue
Continuing.
Sorry, wrong password. Try again!
[Inferior 1 (process 14) exited normally]
(gdb)
```

strcmp trả về eax = 1 (khác 0) vì "hello" khác passphrase, nên lệnh jne nhảy tới 0x8049453 (nhánh báo sai) và chương trình in “Sorry, wrong password. Try again!”. Đây là bằng chứng động khớp với phân tích tĩnh ở Ghidra.

Chạy lại với passphrase đúng. Đặt breakpoint tại lệnh call mix_hash (0x08049436), dùng **si** (step instruction: đi vào bên trong hàm được gọi) và **finish** (chạy đến khi hàm hiện tại trả về) để xem khoá giải mã mà mix_hash trả về trong eax:

```
(gdb) delete
(gdb) break *0x08049436
(gdb) run
(gdb) si
(gdb) x/3i $pc
(gdb) finish
(gdb) p/x $eax
(gdb) continue
```

```
student@ubuntu: ~/lab4
(gdb) delete
Delete all breakpoints? (y or n) y
(gdb) break *0x08049436
Breakpoint 2 at 0x8049436
(gdb) run
Starting program: /home/student/lab4/basic-reverse
Supported authentication methods:
1. Hard-coded password
2. A group of numbers
3. Username/password
Enter your choice: 1
Enter the hard-coded password (option 1):
UIT-NT209-Hello-Ghidra
Congratulation! You got the hard-coded password!

Breakpoint 2, 0x08049436 in hardCode ()
(gdb) si
0x080492b6 in mix_hash ()
(gdb) x/3i $pc
=> 0x80492b6 <mix_hash>:        endbr32
   0x80492ba <mix_hash+4>:      push   ebp
   0x80492bb <mix_hash+5>:      mov    ebp,esp
(gdb) finish
Run till exit from #0  0x080492b6 in mix_hash ()
0x0804943b in hardCode ()
(gdb) p/x $eax
$2 = 0xa3043bc2
(gdb) continue
Continuing.
Your flag: NT209{h4rdc0d3d_str1ngs_4r3_n0t_s3cr3ts}
[Inferior 1 (process 18) exited normally]
(gdb) quit
student@ubuntu:~/lab4$
```

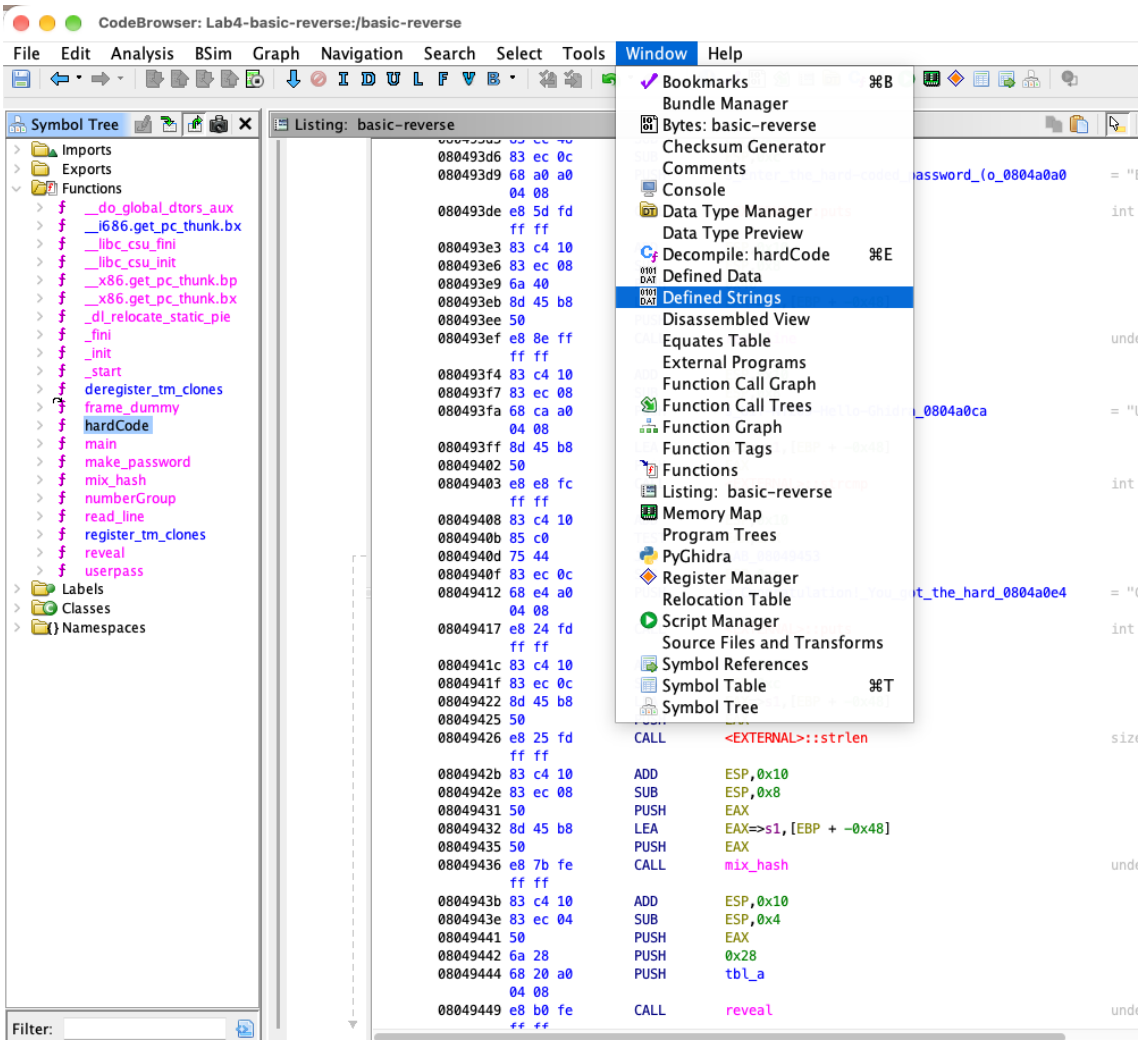
Sau finish, chương trình quay về hardCode tại 0x0804943b, eax = 0xa3043bc2 là khoá được tính từ input đúng. Khoá này được truyền cho reveal để giải mã flag. Lệnh continue chạy tiếp đến hết và in flag của option 1. Thoát GDB bằng quit.

Sinh viên áp dụng quy trình tương tự (Ghidra để hiểu thuật toán, GDB để kiểm chứng giá trị lúc chạy) cho hàm numberGroup (option 2) và userpass (option 3). Xem các gợi ý ở mục C.

C. MỘT SỐ GỢI Ý PHÂN TÍCH VỚI GHIDRA VÀ GDB

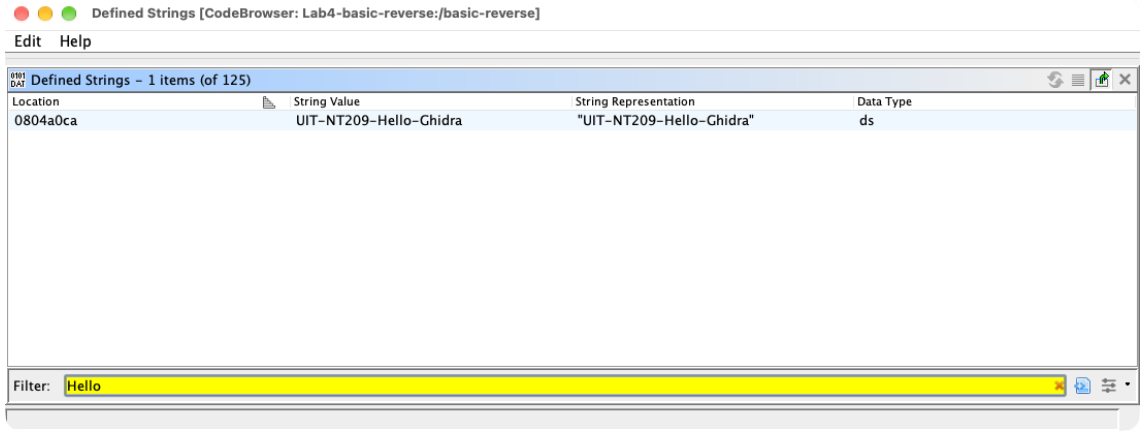
C.1 Các cửa sổ hữu ích của Ghidra

Menu **Window** của CodeBrowser mở thêm các cửa sổ: Defined Strings, Function Graph, Functions, Bytes, Symbol References...

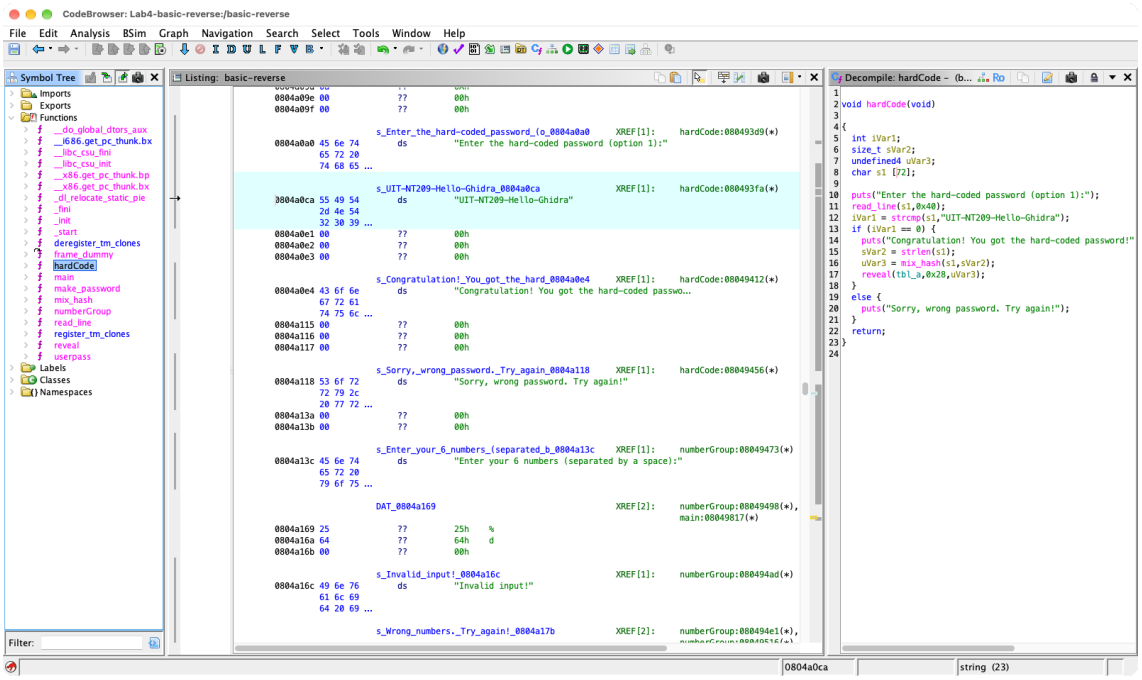


- **Defined Strings và tham chiếu chéo (XREF)**

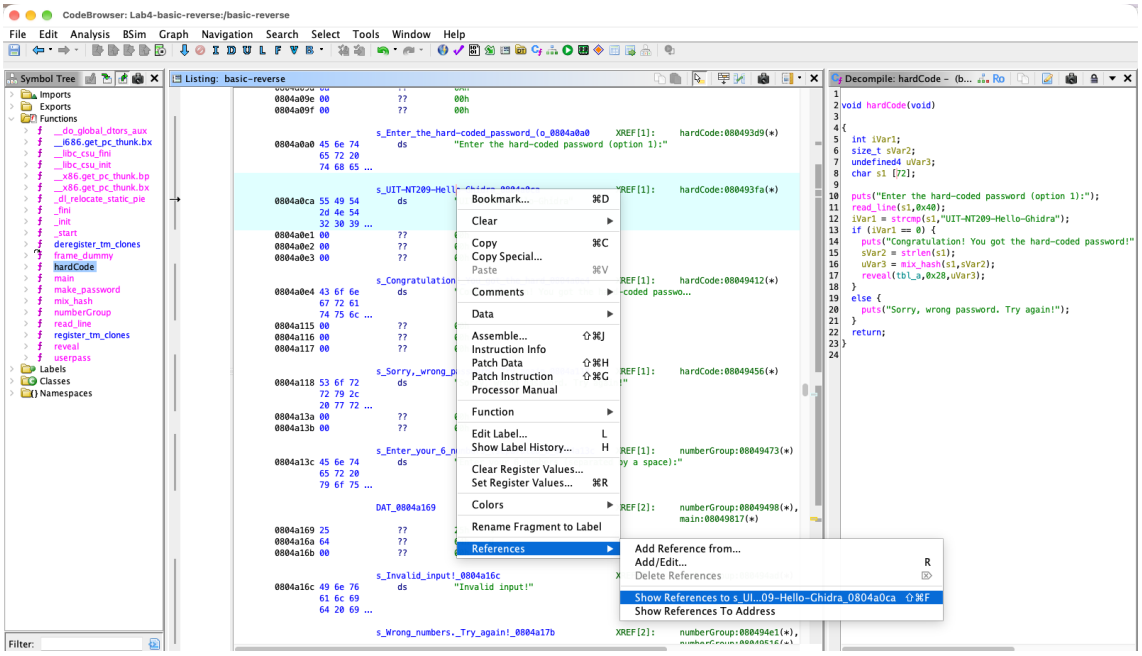
Chọn **Window > Defined Strings** để liệt kê các chuỗi trong file. Ô Filter ở dưới giúp lọc nhanh, ví dụ gõ Hello:



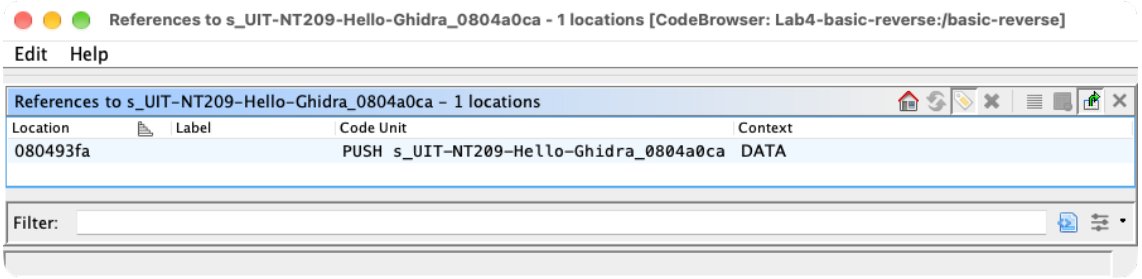
Nhấp đúp vào chuỗi để nhảy tới vị trí trong Listing. Dòng **XREF[1]: hardCode:080493fa(*)** cho biết chuỗi được tham chiếu bởi lệnh tại 0x080493fa trong hàm hardCode (chính là lệnh push 0x804a0ca trước strcmp). Nhấp đúp vào XREF để nhảy đến lệnh đó.



Cách khác: nhấp chuột phải vào địa chỉ hoặc tên chuỗi, chọn **References > Show References to Address** (phím tắt Ctrl+Shift+F):

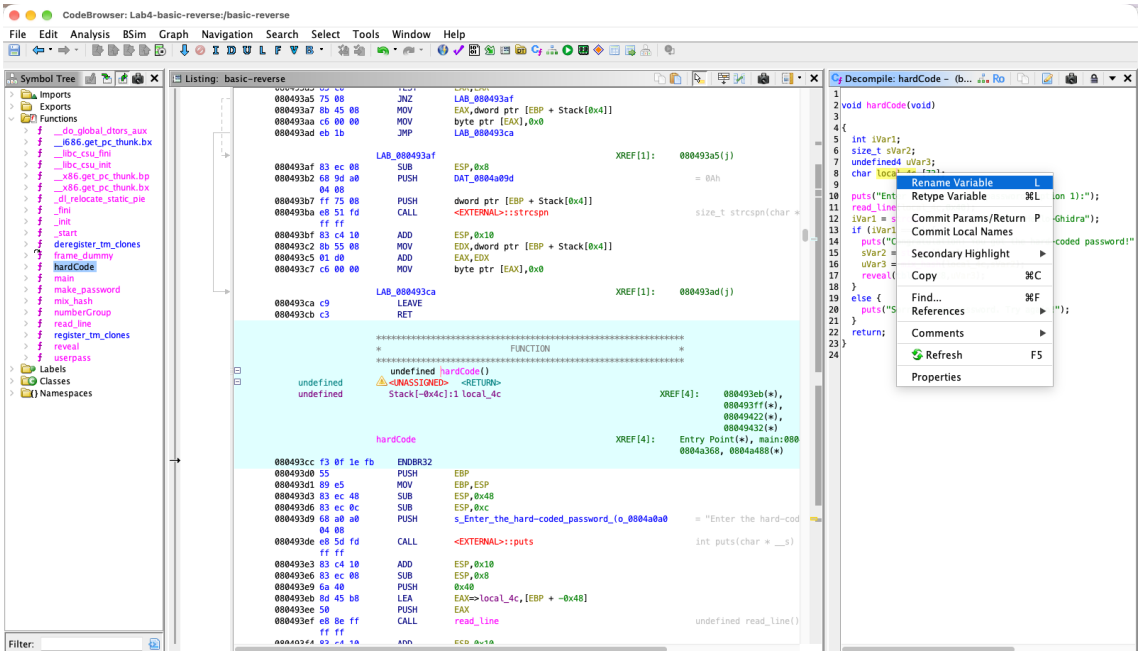


Cửa sổ References liệt kê mọi nơi dùng địa chỉ này. Cách này rất hữu ích để từ một thông báo như "Congratulation" hay "Your flag" lần ngược về đoạn code kiểm tra input.

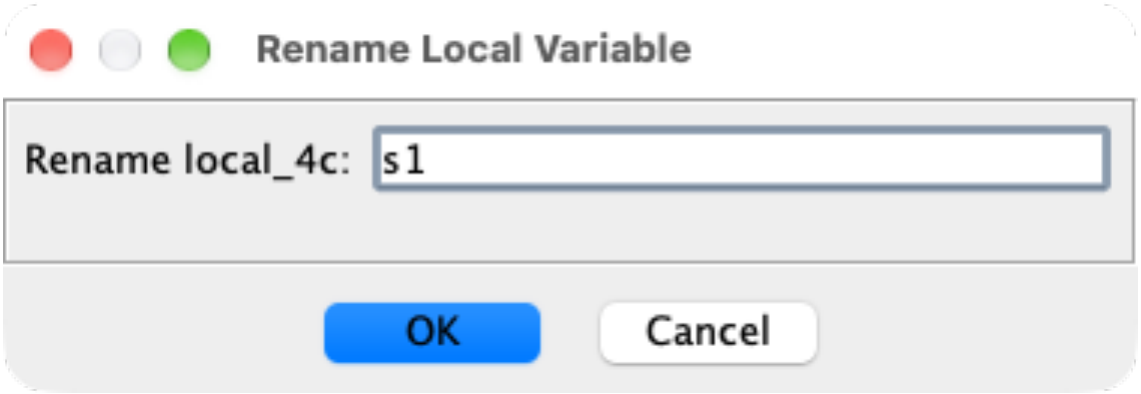


- **Đổi tên biến trong hàm**

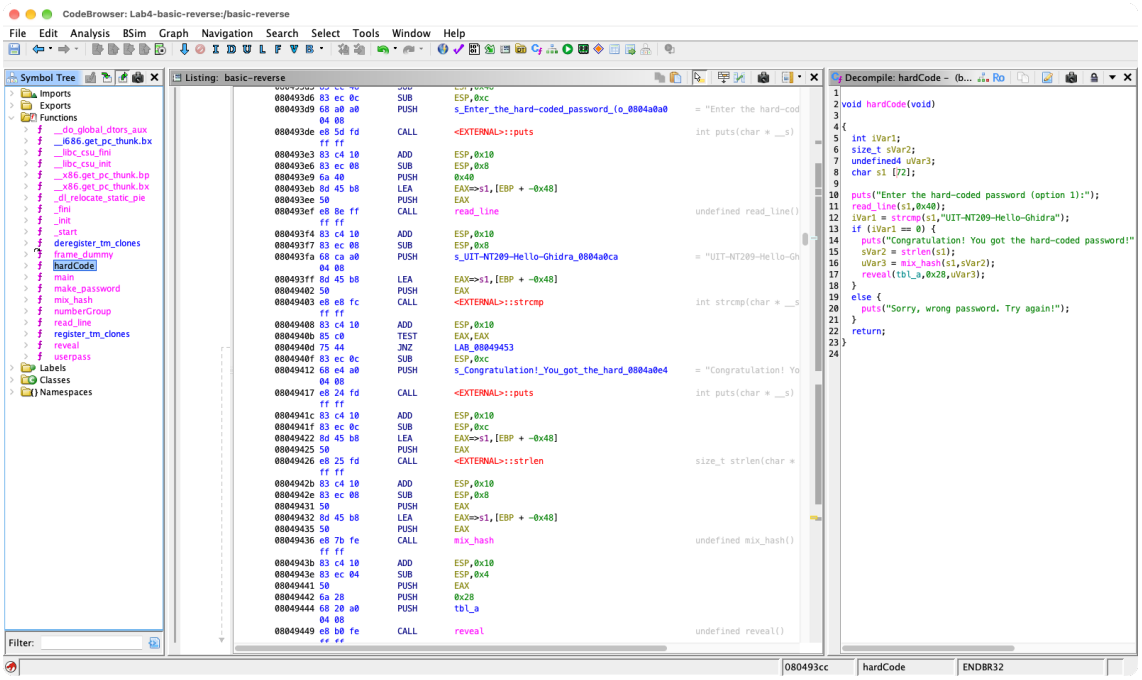
Trong hardCode, nhấp chuột phải vào local_4c, chọn **Rename Variable**:



Đặt tên mới, ví dụ **s1** (hoặc input) rồi bấm OK:

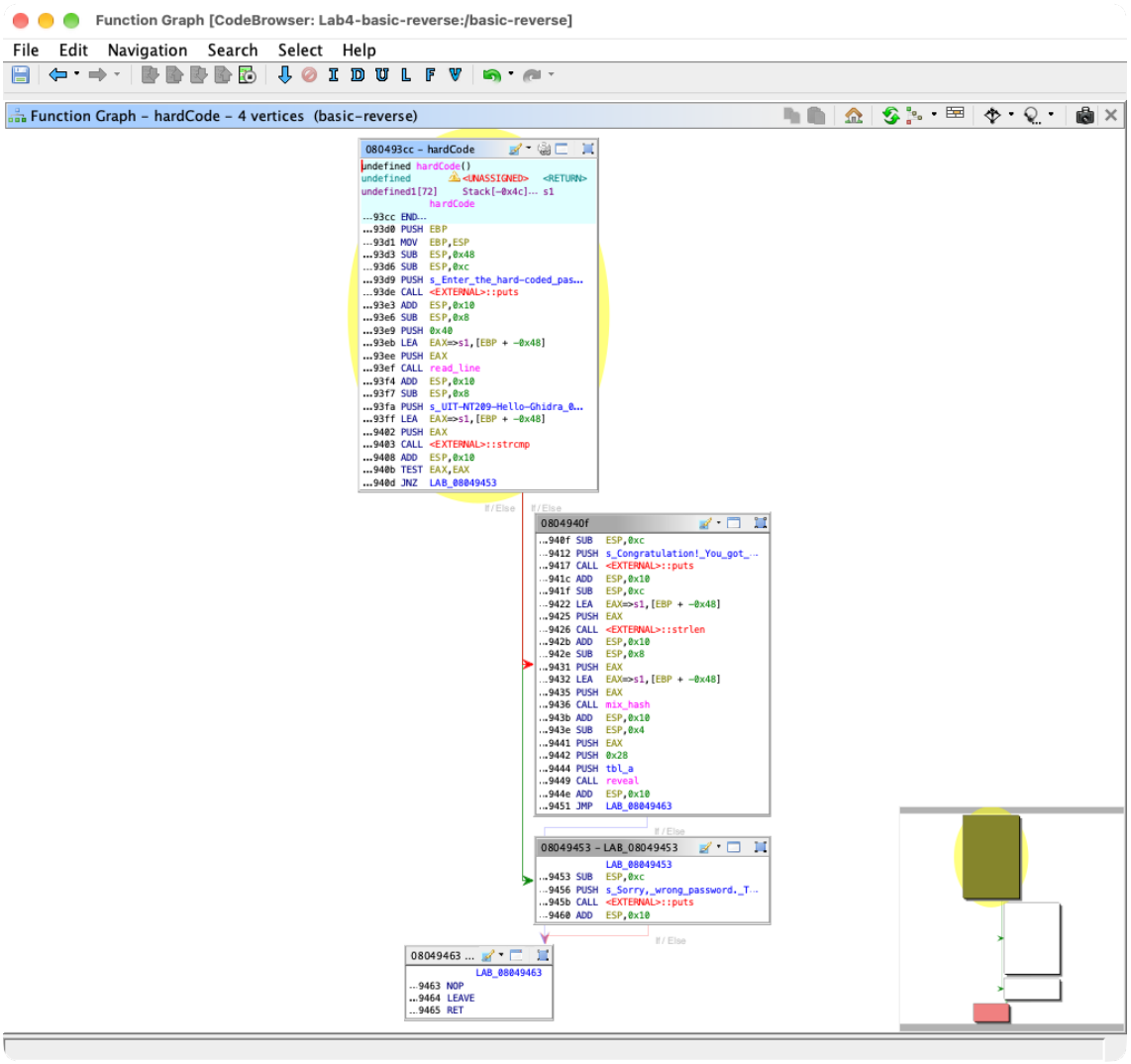


Tên mới được cập nhật ở cả Decompile và Listing. Đặt tên có nghĩa cho biến và hàm là thói quen quan trọng khi dịch ngược hàm dài.



- **Function Graph**

Window > Function Graph vẽ hàm hiện tại dưới dạng các khối lệnh (basic block) nối bằng mũi tên rẽ nhánh: mũi tên xanh lá là nhánh nhảy khi điều kiện đúng, đỏ là khi điều kiện sai. Với hardCode thấy rõ hai nhánh sau strcmp: thành công và báo sai.



C.2 Các lệnh GDB thường dùng

+ **set disassembly-flavor intel**: hiển thị cú pháp Intel (giống Ghidra).
+ **info functions [regex]**: liệt kê hàm; **disassemble <hàm>**: dịch ngược một hàm.
+ **break *<địa chỉ>** / **break <hàm>**: đặt breakpoint; **info breakpoints**, **delete [số]**: xem / xoá.
+ **run**: chạy chương trình; **continue** (c): chạy tiếp đến breakpoint sau.
+ **ni** / **si**: chạy 1 lệnh assembly (ni bước qua lời gọi hàm, si đi vào hàm); **finish**: chạy đến khi hàm hiện tại trả về.
+ **info registers** (i r), **p $eax**, **p/x $eax**: xem thanh ghi.
+ **x/s**, **x/wx**, **x/dw**, **x/i**: xem bộ nhớ dạng chuỗi, word hex, word thập phân, lệnh. Ví dụ x/4dw <địa chỉ> in 4 số nguyên 4 byte liên tiếp.
+ **x/s *(char)$esp: in chuỗi mà tham số thứ nhất (trên đỉnh stack, ngay trước lệnh call) trỏ tới.**

C.3 Gợi ý cho các Yêu cầu

- **Option 2 – numberGroup**
  + Xác định hàm đọc bao nhiêu số và lưu vào đâu (mảng số nguyên trên stack). Trong Ghidra nên Retype biến mảng thành int[n] để mã giả hiển thị dạng a[i].
  + Liệt kê từng điều kiện kiểm tra (các lệnh cmp và lệnh nhảy theo sau, hoặc các câu lệnh if trong Decompile): điều kiện cho phần tử đầu, điều kiện giữa các phần tử kế tiếp trong vòng lặp. Từ đó suy ra dãy số thoả mãn.
  + Kiểm chứng bằng GDB: đặt breakpoint tại lệnh cmp trong vòng lặp, chạy với dãy thử, dùng p $eax hoặc x/<n>dw <địa chỉ mảng> để xem giá trị thực tế được so sánh.
  + Chú ý mọi ràng buộc, kể cả ràng buộc trên từng phần tử riêng lẻ: chúng quyết định đáp án có duy nhất hay không.
- **Option 3 – userpass**
  + Hàm đọc username và password, kiểm tra độ dài, sau đó gọi một hàm phụ để sinh ra password mong đợi từ username rồi so sánh bằng strcmp.
  + Cách tĩnh: đọc hàm phụ trong Decompile, viết lại thuật toán bằng Python/C để tính password cho username của nhóm.
  + Cách động: đặt breakpoint ngay tại lệnh call strcmp trong userpass, chạy với username của nhóm và một password bất kỳ có độ dài hợp lệ, rồi dùng x/s *(char **)$esp và x/s *(char**)($esp+4) để xem hai chuỗi đang được so sánh.
  + Chú ý các điều kiện về độ dài username/password trước khi tới strcmp: nếu không thoả, chương trình báo sai sớm và breakpoint không bao giờ được chạm tới.
  + Báo cáo phải trình bày thuật toán sinh password (không chỉ chụp kết quả GDB).

C.4 Tham khảo thêm

C.4.1 Truyền tham số cho hàm trong assembly IA32

Một số lưu ý về hàm trong mã assembly IA32 như sau:

- Một hàm được gọi với lệnh **call <tên hàm>**
- Một hàm có thể có các tham số truyền vào, các tham số này thường được truyền bằng những câu lệnh **push** giá trị trước lệnh **call** theo thứ tự ngược với khai báo trong C.

Ví dụ một hàm C có tên **add(int a, int b)** thì trong assembly sẽ tìm thấy đoạn mã:

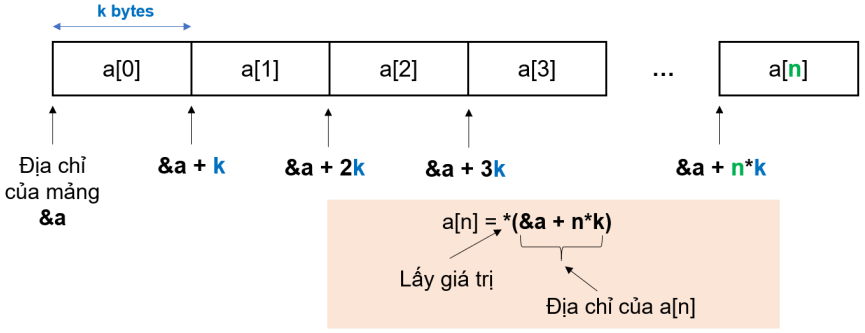
```
push b
push a
call add
```

- Sau khi được gọi với lệnh call, nếu hàm có trả về giá trị thì thường lưu trong %eax.

C.4.2 Mảng

Trong hệ thống, các phần tử của 1 Mảng (array) được đặt ở các vị trí liền kề nhau trong bộ nhớ. Trong 1 mảng, ta luôn có:

- Phần tử đầu tiên (index = 0) luôn nằm ở ô nhớ có địa chỉ thấp nhất so với các phần tử còn lại, sau đó đến phần tử thứ 2, thứ 3,... lần lượt đến phần tử cuối của mảng.
- Vị trí của các phần tử nằm cách nhau 1 khoảng bằng kích thước của mỗi phần tử. Ví dụ, một mảng ký tự có các phần tử nằm cách nhau 1 byte, các phần tử của mảng số nguyên (integer) cách nhau 4 byte.



D. YÊU CẦU & ĐÁNH GIÁ

Sinh viên thực hành và nộp bài theo **nhóm tối đa 3 sinh viên**. Các nhóm báo cáo kết quả thực hành theo 2 bước:

- **Bước 1:** Submit đáp án (input và flag NT209{...}) tìm được của các Yêu cầu trên **Hệ thống online** theo hướng dẫn của GVTH.
- **Bước 2:** Nộp file báo cáo kết quả **(.pdf)** trình bày cách phân tích file và tìm kiếm các input cho các option bao gồm:
  - Mô tả *cách thức* kiểm tra của mỗi hàm xử lý (phép so sánh, thuật toán) để tìm ra input tương ứng với 3 option.
    - Option 1 yêu cầu phân tích bằng mã assembly (Listing của Ghidra hoặc lệnh disassemble của GDB), thể hiện rõ các dòng lệnh truyền tham số, gọi hàm, tính toán và kiểm tra giá trị trả về. *Có thể bỏ qua các đoạn mã tạo và thu hồi stack frame.*
    - Option 2 và 3: Sinh viên có thể dựa trên mã giả (cửa sổ Decompile của Ghidra) để hiểu hoạt động, cần phân tích, mô tả các đoạn code quan trọng.
    - Các đoạn code (C/Python...) để hỗ trợ tìm input nhanh/tự động (có giải thích).

- Hình ảnh chụp màn hình kết quả thực thi file **basic-reverse** với input đã tìm được (thấy rõ flag in ra) cho 3 option.
- Flag của từng option kèm hình ảnh chương trình in ra flag đó. Flag phải khớp với input đã nộp.
- Định dạng file **Lab4_NhomX_MSSV1-MSSV2-MSSV3.pdf**

E. THAM KHẢO

[1] Randal E. Bryant, David R. O'Hallaron (2011). *Computer System: A Programmer's Perspective (CSAPP)*

[2] National Security Agency, Ghidra – Getting Started [Online]. https://github.com/NationalSecurityAgency/ghidra/blob/master/GhidraDocs/GettingStarted.md

[3] National Security Agency, Ghidra Cheat Sheet [Online]. https://ghidra-sre.org/CheatSheet.html

[4] Free Software Foundation, Debugging with GDB [Online]. https://sourceware.org/gdb/current/onlinedocs/gdb/

HẾT